Julian Steinberg
117212549

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i := a.Length-1;
    out := 0;
    while (i >= 0) {
        out := out - a[i];
        i := i-1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i := 0;
    out := 0;
    while (i < a.Length) {
        out := out - a[i];
        i := i + 1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a` a → a

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b → (a, Maybe b)

(d) `\x l -> x : l ++ l ++ [x]`
a → b → [a,b]

(e) `foldr (const 1)` "one"
ill-typed

(f) (,"42")
Nothing, String

(g) (,) "42"
Ill-typed

(h) `reverse . reverse`
Ill-typed

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
a → Bool

(k) `fmap (fmap (+1)) [Nothing]`
Ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers.

```
\x -> x + 1
(+1)
```

(b) `Bool -> [Bool]`

```
\x -> [x == False]
```

(c) `a -> Maybe b`

```
\x -> if x+1 == 2 then show True else show Nothing
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
zipWith 1 "a" True
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
zipWith 1 "a" True
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
(a -> \x y z =
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

```
((x,y) == True)
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ y = m*m*m                                  }}
  y := 0;
{{ y >= 0 && m == m+0                         }}
  var x := m * m;
{{ x == m*m                                   }}
  var z := m;
{{ true                                       }}
  while (z > 0) {
{{ true                                       }} ->>
{{ z >= 0 && y >= x                           }}
    z := z - 1;
{{ true                                       }}
    y := y + x;
{{ true                                       }}
  }
{{ z >= 0 && m >= 0 && m == m                 }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
Weave :: [a] -> [a] -> [a]
Weave [] [] = []
Weave xs [] = xs
Weave [] ys = ys
Weave (x:xs) (y:ys) = [x,y] (++) Weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
Powerset :: [a] -> [[a]]
Powerset [] = [[]]
Powerset (x:xs) = [x:xs] ++ [x] ++ [xs] ++ Powerset xs
```

Stephanie Farace

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    out := 0;
    var i : int := a.Length;
    while i >= 0 {
      out := out - a[i]
      i := i - 1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    out := 0
    var i : int := 0;
    while i < a.Length - 1 {
      out := out - a[i];
      i := i + 1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → Maybe a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [b] →

(e) `foldr (const l)`
a → [b] → [b]

(f) `(,"42")`
~~String → String → String~~ ill-Typed

(g) `(,) "42"`
a → String → (a, String)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(b → a) → a → bool

(k) `fmap (fmap (+1)) [Nothing]`
(a → b) → Maybe a → Maybe b

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`Foo a = not a : []`

(c) `a -> Maybe b`
undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`Foo f x y = f (head x) (head y)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`Foo f x y = (f (Just x) (Just y))`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`Foo f g x = foldr (\a acc -> if g (f a) then f a : acc else acc) x []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`Foo a f = zip (maybeToList(a)) (f a)`

(h) `Eq a => a -> [a] -> [a]`
delete

(i) `Show a => [a] -> IO String`
`Foo a = foldr (\a = show a) a " "`

(j) `(a,b) -> (a -> b -> c) -> c`
`Foo x f = case x of`

(k) `((a,b) -> c) -> c`
`Foo x = x (5, "hello")`
`(a, b) -> f a b`
`otherwise -> null`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with `->>`. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m*m*(0)
  y := 0;
{{ y = m*m*(m-m)
  var x := m * m;
{{ y = x*(m-m)
  var z := m;
{{ y = x*(m-z)
  while (z > 0) {
{{ y = x*(m-z) && m>=0 && z>0
{{ y+x = x*(m-(z-1)) && m>=0
    z := z - 1;
{{ y+x = x*(m-z) && m>=0
    y := y + x;
{{ y = x*(m-z) && m>=0
}
{{ y = x*(m-z) && m>=0 && !(z>0)
{{ y = m * m  * m}}
```

}} $0 = m*m*(m-m)$

}} $y = m*m*(m-m)$

}} $y = x*(m-m)$

}} $y = x*(m-z)$

}} ->> $y = x*(m-z) \;\&\&\; m>=0 \;\&\&\; z>0$

}} $y+x = x*(m-(z-1)) \;\&\&\; m>=0$

}} $y+x = x*(m-z) \;\&\&\; m>=0$

}} $y = x*(m-z) \;\&\&\; m>=0$

}} ->> $y = x*(m-z) \;\&\&\; m>=0 \;\&\&\; !z>0 \;\&\&\; z>=$

	m	y	x	z
0	m	0	m*m	m
1	m	x	m*m	m-1
2	m	2x	m*m	m-2

	m	y	x	z
0	3	0	9	3
1	3	9	9	2
2	3	18	9	1
3	3	27	9	0

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) = x : y : weave xs : weave ys : []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset [x] = [x] : [[]]
powerset (x:xs) = [x] : [xs] : powerset [xs]
```

~~powerset x = foldr (x acc -> foldr (y acc ->~~

Ethan Jew
UID: 117177541

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i:int := 0;
    out := 0;

    while (i < a.Length) {
        out := out - a[i];
        i := i+1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i:int := a.Length - 1;
    out := 0;

    while (i >= 0) {
        out := a[i] - out;
        i := i-1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
ill-typed

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const 1)`
b -> [b] -> b

(f) `(.) "42"`
a -> (a, String)

(g) `(.) "42"`
a -> (a, String)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
a -> (a, a)

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\b -> if b then [b] else []`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f i c -> case (i, c) of (x:xs, y:ys) -> if y == 'a' then ((f (x+1) y) : [True]) else [False]`
`[] -> [True]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a1 b1 -> case (a1, b1) of (Just a, Just b) -> maybe (f a b)`
`_ -> Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 lst -> case lst of (x:xs) -> if f2 (f1 a) then [f1 a] else []`
`[] -> []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\a1 f -> case a1 of maybe a -> case (f a) of (x:xs) -> [(a,x)] [] -> []`
`nothing -> []`

(h) `Eq a => a -> [a] -> [a]`
`\a lst -> a : lst`

(i) `Show a => [a] -> IO String`
`\a -> putStr a`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`\(a,b) c -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

X Y Z
m×m 0 m
m×m m-1
(m×m)×2 m-2

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == m*m*(m-m) && m≥0 && m*m == m*m                    }}
  y := 0;
{{ y == m*m*(m-m) && m≥0 && m*m == m*m                    }}
  var x := m * m;
{{ y == m*m*(m-m) && m≥0 && x == m*m                      }}
  var z := m;
{{ y == m*m*(m-z) && z≥0 && x == m*m                      }}
  while (z > 0) {
{{ y == m*m*(m-z) && z≥0 && x == m*m && z>0               }} ->>
{{ y+x == m*m*(m-z+1) && z-1 >= 0 && x == m*m             }}
    z := z - 1;
{{ y+x == m*m*(m-z) && z>=0 && x == m*m                   }}
    y := y + x;
{{ y == m*m*(m-z) && z >= 0 && x == m*m                   }}
  }
{{ y == m*m*(m-z) && z>=0 && !(z>0)                       }} ->>
{{ y = m * m  * m}}
```

x == m*m &&

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
f lst1 lst2 = case (lst1, lst2) of
                (x:xs, y:ys) -> (x : y : []) ++ (f xs ys)
                (_ , _)      -> []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
f lst = case lst of
          (x:xs) -> f' x xs (length xs) ++ f xs
          []     -> []
```

```
f' head lst len i = if i < len then
                      lst : (case lst of
                               (x:xs) -> f' x (xs ++ head) len (i+1)
                               []     -> []
                            )
                    else
                      []
```

Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*>) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence  :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...
```

1 Hoare Rules

```
  { P } s1 { Q }
  { Q } s2 { R }
------------------- (hoare_seq)
{ P } s1; s2 { R }


----------------------- (hoare_asgn)
{Q [x := a]} x := a {Q}


  {P'} c {Q'}
P ==> P'    Q' ==> Q
-------------------- (hoare_consequence)
   {P} c {Q}

         {P &&  b} s1 {Q}
         {P && !b} s2 {Q}
-------------------------------- (hoare_if)
{P} if b { s1 } else { s2 } {Q}

         {P && b} s {P}
---------------------------- (hoare_while)
{P} while b { s } {P && !b}
```

Prelude Functions and Datatypes

Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not  :: Bool -> Bool
```

Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null   :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl  :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr  :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or  :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

Typeclass Definitions

```
class Semigroup a where
  (<>)   :: a -> a -> a

class Semigroup m => Monoid m where
  mempty :: m

class  Show a  where
  show   :: a   -> String

class  Eq a  where
  (==)   :: a -> a -> Bool
  (/=)   :: a -> a -> Bool

class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min             :: a -> a -> a

class Functor f where
  fmap   :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
  pure   :: a -> f a
  (<*>)  :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b

class Foldable t where
  foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
  arbitrary :: Gen a
  shrink    :: a -> [a]

class Num a where
  (+), (-), (*)       :: a -> a -> a
  negate              :: a -> a
  abs                 :: a -> a
  signum              :: a -> a
  fromInteger         :: Integer -> a
```

Angela Cheng

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

[1,2,3] -> 1-(2-(3-0))

[1,2,3] -> 3-(2-(1-0))

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var x: int := a.Length-1;
  out:=0;
  while (x>=0){
    out:= a[x] - out;
    x:= x-1;
  }
  return out;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var x: int := 0;
  out:=0;
  while (x<a.Length){
    out:= a[x]-out;
    x:=x+1;
  }
  return out;
}
```

const :: a -> b -> b

(,) :: a -> b -> (a,b)

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String
show :: a -> String
== :: a -> a -> Bool

(d) `\x l -> x : l ++ l ++ [x]`
a -> [b] -> [c]

(e) `foldr (const 1)` foldr :: (a -> b -> b) -> b -> t a -> b
b -> [a] -> b

(f) (,"42")
ill-typed

(g) (,) "42"
b -> (a,b)

(h) `reverse . reverse`
reverse :: [a] -> [a]
[a] -> [b] -> [c]

(i) `\l -> filter (< 42) (l ++ l)`
filter :: (a -> Bool) -> [a] -> [a]
(++) :: [a] -> [a] -> [a]
[a] -> [a]

(j) `let f x = x in (f 'a', f True)`
a -> (char, bool)

(k) `fmap (fmap (+1)) [Nothing]`
fmap : (a -> b) -> f a -> f b
[int] -> [Maybe ints]

(.) :: (b,c) -> (a,b) -> a -> c

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
repeat True

(c) `a -> Maybe b`
\x -> Nothing

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
exprd (x:xs) (y:ys) = if x==y.toInt then True:(exprd xs ys) else False:(exprd xs ys)

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
map (+1) (takeWhile (>3) lst)

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
replicate(maybeToList lst)

(h) `Eq a => a -> [a] -> [a]`
delete x lst

(i) `Show a => [a] -> IO String`
\xs -> putStr (head xs) ← I forgot if it's putStr or putStrLn

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m≥0 ∧ 0 = m·m·m                                  }}
  y := 0;
{{ m≥0 ∧ y = m·m·m                                  }}
  var x := m * m;
{{ m≥0 ∧ y = x·m·m                                  }}
  var z := m;
{{ z≥0 ∧ y = x·m                                    }}
  while (z > 0) {
{{ z≥0 ∧ y = x·m ∧ z>0                              }} ->>
{{ z-1≥0 ∧ y+x = x·m                                }}
    z := z - 1;
{{ z≥0 ∧ y+x = x·m                                  }}
    y := y + x;
{{ z≥0 ∧ y = x·m                                    }}
}
{{ z≥0 ∧ y = x·m ∧ ¬(z>0)                           }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset lst = case lst of
  | -- iterate through list
  | otherwise = [[]]
```

Julian Jarillo

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;

  while i >= 0 {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;

  while i < a.Length {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
Int -> Int -> [Int]

(e) `foldr (const 1)`
Foldable t => (Int -> b -> Int) -> b -> t Int -> b

(f) `(."42")`
String -> b -> [String, b)

(g) `(.) "42"`
String -> b -> [String, b)

(h) `reverse . reverse`
([a] -> [a]) -> ([a] -> [a]) -> (a)

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
((a -> a) -> a) -> ((String -> String), (Bool -> Bool))

(k) `fmap (fmap (+1)) [Nothing]`

((Int -> Int -> Int) -> [Int] -> [Int]) -> [Maybe Int] -> [Int]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [!x]`

(c) `a -> Maybe b`
`\x -> Just x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f x y -> if (head x == 1 && head y == 'a') && (f (head x) (head y)) then [True] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f x y -> Maybe (f (Just x) (Just y))`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f g x -> if g (f (head x)) then [f (head x)] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x f -> [(Just x, head (f (Just x)))]`

(h) `Eq a => a -> [a] -> [a]`
`\x (y:ys) -> if x==y then [x] else ys`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`\(\(x,y) -> z) -> z`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m=3

y	x	z	m
0	9	3	3
9	9	2	3
18	9	1	3
27	9	0	3

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0=0 ∧ m·m = m·m                          }}
  y := 0;
{{ y = 0 ∧ m·m = m·m                        }}
  var x := m * m;
{{ y = x·0 ∧ x = m·m                        }}
  var z := m;
{{ y = x·(m-z) ∧ x = m·m                    }}
  while (Z > 0) {
{{ y = x·(m-z) ∧ x = m·m ∧ z > 0             }} ->>
{{ y+x = x·(m-z-1) ∧ x = m·m                 }}
    z := z - 1;
{{ y+x = x·(m-z) ∧ x = m·m                   }}
    y := y + x;
{{ y = x·(m-z) ∧ x = m·m                     }}
}
{{ y = x·(m-z) ∧ x = m·m ∧ !(z > 0)          }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = (x:y) ++ (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = []
powerset (x:xs) = (x:xs) : (powerset xs ++ [x])
```

Austin Nguyen ~ 17159257

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.Length
  while (i >= 0) {
    out := out + a[i]
    i := i-1
  }
  return out
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i:int := 0
  while (i < a.Length) {
    out := out - a[i]
    i := i+1
  }
  return out
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`

ill-typed

(d) `\x l -> x : l ++ l ++ [x]`

a → [a] → [a]

(e) `foldr (const l)`

a → [a] → a

(f) `(,"42")`

(a, String)

(g) `(,) "42"`

(String, b)

(h) `reverse . reverse`

[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`

[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`

char → (char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`

fmap int → fmap int

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [x]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\x y -> zipWith (;) [1] ['c']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x y -> Maybe x = Nothing | Maybe y = Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\x -> map`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`
`\x -> rmb x`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`
`\`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m'm= m'.m'(m-1)                                  }}
  y := 0;
{{ y+m'm= m'm'(m-1)                                 }}
  var x := m * m;
{{  y+x= x'(m-1)                                    }}
  var z := m;
{{  y+x = x*(z-1)                                   }}
  while (Z > 0) {
{{  y+x = x*(z-1) && z>0                            }} ->>
{{  y+x= x*(z-1)                                    }}
    z := z - 1;
{{  y+x= x*z                                        }}
    y := y + x;
{{  y=x*z                                           }}
}
{{ y=x*z && !(z>0)                                  }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs)(y:ys) =
    x : y : weave (xs)(ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset (x:xs) acc =
  acc=[]
  powerset xs x(x:xs): acc
  powerset xs (  xs) : acc
  powersetxs (x) : acc
  powerset xs _ = acc
```

Nayan Brik 117841589

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
var i:=0
while (i < a.length)
invariant 0≤i≤a.length && out<=0
{
    out = out - a[i];
    i = i+1;
}
}
```

```
method f2(a:array<int>) returns (out:int) {
var i:= a.length;
while ( i>= 0 )
invariant i≤ a.length && out<=0
{
    out = out - a[i];
    i = i-1;
}
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)` a -> y -> (a, y)

(c) `\x y -> if x == y then show x else show (x,y)`
show :: a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> a -> [a]

(e) `foldr (const 1)`
[a] -> a

(f) (,"42")
(null, String)

(g) (.) "42"
(null, null) -> String

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
a -> [a]

(j) `let f x = x in (f 'a', f True)`
(Maybe Char, Maybe Bool -> a) -> b -> (a, a)

(k) `fmap (fmap (+1)) [Nothing]`
[Nothing]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [x]`

(c) `a -> Maybe b`
`\x -> Px`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\a b c -> if $and $zipWith a b c than $zipWith a b c else $zipWith a b c`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\a b c -> zipWith (Maybe a) (Maybe b) (Maybe c)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f y z -> filter y (map f z)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x y -> zip (take 5 (repeat x)) (y x))`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`
`\x -> 'show(take 5 (repeat x))`

(j) `(a,b) -> (a -> b -> c) -> c`
`\x y -> where x`

(k) `((a,b) -> c) -> c`
`(a,b) -> y a b`
`_ -> Nothing`

`\x -> x (a,b)`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                    }}
  y := 0;
{{                                                    }}
  var x := m * m;
{{                                                    }}
  var z := m;
{{                                                    }}
  while (z > 0)
{{                                                    }} ->>
  {{                                                  }}
    z := z - 1;
  {{                                                  }}
    y := y + x;
  {{                                                  }}
  }
{{                                                    }} ->>
{{ y = m * m * m}}
```

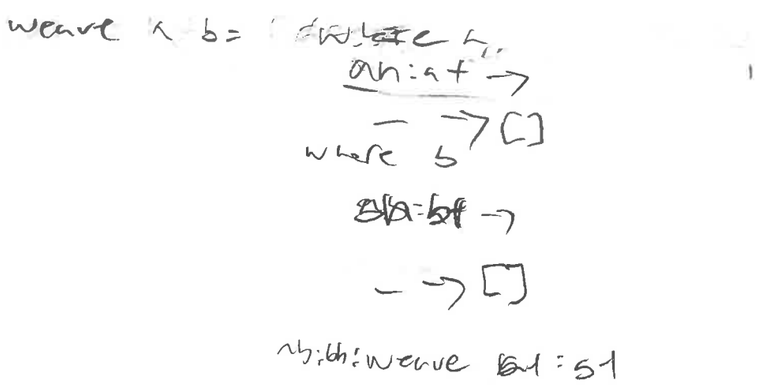
Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

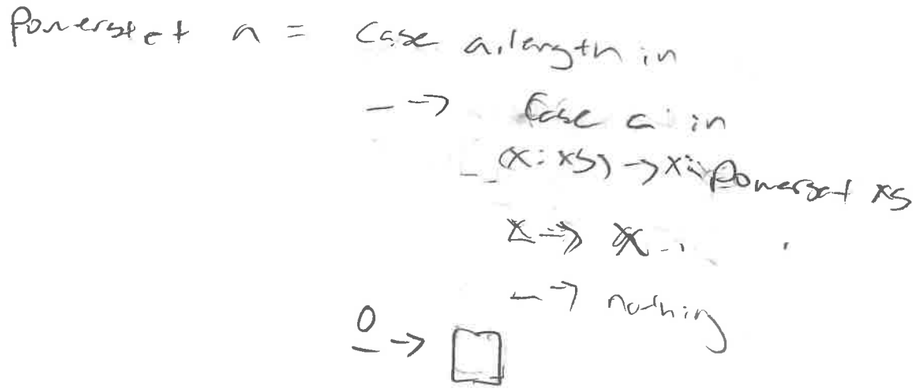
```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.



(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```



Abigail Song
11803487

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while (i >= 0) {
    i := i - 1;
    out := out - a[i];
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer: `a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
int → a → int

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
a → (String, a)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[int] → [int]

(j) `let f x = x in (f 'a', f True)`
(String, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
[Maybe a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

```
if b then b : [True]
```

(c) `a -> Maybe b`

```
a -> Nothing
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
f (a:as) (b:bs) = if f a b then [f a b]
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
\f (Just a) (Just b) = Just f a b
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
undefined
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
f (Just a) = [(a, f a)]
```

(h) `Eq a => a -> [a] -> [a]`

```
delete
```

(i) `Show a => [a] -> IO String`

```
foo [x : xs] = putStrLn (x)
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
undefined
```

(k) `((a,b) -> c) -> c`

```
undefined
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == (m * m)(0)                                  }}
  y := 0;
{{ y == (m * m)(m - m)                              }}
  var x := m * m;
{{ y == x (m - m)                                   }}
  var z := m;
{{ y == x (m - z)                                   }}
  while (Z > 0) {
{{ y == x (m - z)  && z > 0                         }} ->>
{{ y + x == x (m - (z - 1))                         }}
    z := z - 1;
{{ y + x == x (m - z)                               }}
    y := y + x;
{{ y == x (m - z)                                   }}
}
{{ y == x (m - z)  && z <= 0                        }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [b] -> [a, b]
weave list [] = list
weave [] list = list
weave (x:xs) (y:ys) = [x, y] : weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [a]
powerset [] -> []
powerset (x:xs) = \x -> [x] in powerset xs
```

MAXIMILIAN SON 117265187

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

[1, 2, 3, 4]

foldr → 0-1-2-3-4

foldl → 0-4-3-2-1

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i : int := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
  }
}

}
```

```
method f2(a:array<int>) returns (out:int) {
  var i : int := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
  }
}

}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const l)`
a -> [b] -> [a]

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
b -> (String, b)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
(String, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
[Maybe a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\b -> b : [True]`

(c) `a -> Maybe b`
undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f (i:is) (c:cs) -> if i == 0 && c == 'c' then [f i c] else []`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f (Just x) (Just y) -> Just (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 lst -> let (x:xs) = lst in if f2 (f1 x) then [f1 x] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\(Just m) f -> let (x:xs) = f m in [(m, x)]`

(h) `Eq a => a -> [a] -> [a]`
`\x (y:ys) -> if x == y then [x] else [y]`

(i) `Show a => [a] -> IO String`
`\(s:ss) -> putStrLn (show s)`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m times

$m^2 + m^2 + m^2 + \ldots$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

m = 3

```
{{ m > 0 }} ->>
{{ 0 == m x m (0)                                  }}
  y := 0;
{{  y == m x m (m-m)                               }}
  var x := m * m;
{{  y == x (m-m)                                   }}
  var z := m;
{{  y == x (m-z)  && z >= 0                        }}
  while (z > 0) {
{{  y == x (m-z)  && z > 0  && z >= 0              }} ->>
{{  y + x == x (m-(z-1))  && z-1 >= 0              }}
    z := z - 1;
{{  y + x == x (m-z)  && z >= 0                    }}
    y := y + x;
{{  y == x (m-z)  && z >= 0                        }}
}
{{  y == x (m-z)  && z <= 0 && z >= 0              }} ->>
{{ y = m * m  * m}}
```

x = 9

z = 3

z = 2

y = 9

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] l2 = l2
weave l1 [] = l1
weave (x:xs) (y:ys) = [x,y] ++ (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset lst = let (x:xs) = lst in
    lst : [x] : powerset xs
```

Abhilash Sharma
11804104

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Maybe A[i] - out

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.length-1
  out := 0

  while (i ≥ 0) {
    out := a[i] - out
    i := i - 1
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0
  out := 0

  while (i < a.length) {
    out := out - a[i]
    i := i + 1
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const l)`
Foldable T ⇒ b → t a → b

(f) `(."42")`
ill typed

(g) `(.) "42"`
a → (String, a)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
a → a ill typed

(k) `fmap (fmap (+1)) [Nothing]`
ill typed

fmap does n
(a → b) → fa → f b
[Nothing]
does not work

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [True] else [False]`

(c) `a -> Maybe b`
undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\(a,b,c) -> x y -> if c then x+y else`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
undefined

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
undefined

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
undefined

(h) `Eq a => a -> [a] -> [a]`
`\x y -> x:y`

(i) `Show a => [a] -> IO String`
`\x -> (show x)`

(j) `(a,b) -> (a -> b -> c) -> c`
`(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                    }}
  y := 0;
{{                                                    }}
  var x := m * m;
{{                                                    }}
  var z := m;
{{                                                    }}
  while (Z > 0) {
{{                                                    }} ->>
{{                                                    }}
    z := z - 1;
{{                                                    }}
    y := y + x;
{{                                                    }}
}
{{                                                    }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []

weave (x:xs) (y:ys) = x:y:weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) = [[x]]:[xs]: powerset xs
```

Vishal Budamala
11791768

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

(1,2,3,4,5)

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

[1,2,3,4,5] 5-4=1

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int:= a.Length-1;
  out:= 0
  while(i>=0){
    out:= out-a[i];
    i:= i-1;
  }
}

}
```

```
method f2(a:array<int>) returns (out:int) {
  var i:int:=0;
  out:=0;
  while(i< a.Length){
    out:=out-a[i];
    i:=i+1;
  }
}

}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
Bool -> Bool

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [[a]]

(e) `foldr (const 1)`
Foldable t => b -> t a -> b

(f) `(,"42")`
(a -> b -> (a, b))

(g) `(,) "42"`
(a -> b -> (a, b))

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(Int -> Bool) -> Bool

(k) `fmap (fmap (+1)) [Nothing]`
f b

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\x -> [1] -> ['a'] -> [True]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x ->`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\x -> fx -> [a]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
undefined

(h) `Eq a => a -> [a] -> [a]`
`(:)`

(i) `Show a => [a] -> IO String`
undefined

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) -> (a b) -> b`

(k) `((a,b) -> c) -> c`
`\((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m=3
y=0 9 18 27
x=9 9 9 9 y=m×m
z=3 2 1 0

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0==(m*m)*m && (z>=0)                                  }}
  y := 0;
{{ y==(m*m)*m && (z>=0)                                  }}
  var x := m * m;
{{ y==x*m && (z>=0)                                      }}
  var z := m;
{{ y==x*z && (z>=0)                                      }}
  while (z > 0) {
{{ y==x*z && (z>0) && (z>=0)                             }}
{{ y+x==x*(z-1) && (z>=0)                                }} ->>
    z := z - 1;
{{ y+x==x*z && (z>=0)                                    }}
    y := y + x;
{{ y==x*z && (z>=0)                                      }}
  }
{{ (z>=0) && y==x*z && !(z>0)                            }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [Int] -> [Int] -> [Int]
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = (x:y): weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

~~powerset [a] = [[a]]~~

```
Powerset [] = []
Powerset (x:xs) = (x) : powerset xs
```

Bardia Afkhali 117909982

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.Length - 1;
  out := 0;
  while (i >= 0){
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
(a → b → Bool) → a → (a,b)

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [[a]]

(e) `foldr (const 1)`
(a → b → b) → b → ((a → b → b) → a) → b

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
a → b → (a,b)

(h) `reverse . reverse`
not needed?
([a] → [a]) → ((b → c) → (a → b) → a → c) → ([c] → [c])

(i) `\l -> filter (< 42) (l ++ l)`
(Int → Bool) → [Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
\x → Bool && Char

(k) `fmap (fmap (+1)) [Nothing]`
(Int → [Maybe]) → Int → [Maybe]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
True : [False, False]

(c) `a -> Maybe b`
\x -> Nothing || Just y

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
zipWith 3 'a' False

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
zipWith Maybe

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
foldr (filter p xs) 0 xs

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`
delete 1 [1,2,3]

(i) `Show a => [a] -> IO String`
putStrLn "Hello"

(j) `(a,b) -> (a -> b -> c) -> c`
(\(x,y) -> x y z) -> z

(k) `((a,b) -> c) -> c`
\(\(x, y) -> z) -> z

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules-- write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m * m * m  && m > 0                    }}
  y := 0;
{{ y = m * (m * m)  && m > 0                  }}
  var x := m * m;
{{ y = m * x  && m > 0                        }}
  var z := m;
{{ y = z * x  && z > 0                        }}
  while (z > 0) {
{{ y = z * x  && z > 0  && z > 0               }} ->>
{{ (y + x) = (z - 1) * x  && (z - 1) >= 0      }}
    z := z - 1;
{{ (y + x) = z * x  && z >= 0                 }}
    y := y + x;
{{ y = z * x  && z > 0                        }}
  }
{{ y = z * x  && z = 0                        }} ->>
{{ y = m * m * m }}
```

$y = z * x \;\&\&\; z > 0$

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = x:y : weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

n = size of list $2^3 = 8$

```
powerset _ = [ ]
powerset ((x:xs) : xss) = powerset (transpose (x:xs) xss )
```

Nishanth Allam
10018893

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    out := 0;
    var i := a.length-1;
    while (i >=0) {
        out = out - a[i]
        i = i-1;
    }

/*  [1, 2,3,4]
    -> 0 - 4 - 3 - 2 - 1 */
}
```

```
method f2(a:array<int>) returns (out:int) {
    out := 0;
    var i := 0;
    while (i < a.length) {
        out = out - a[i];
        i = i+1;
    }

/* [1,2,3,4]
   -> 0 - 1 - 2 - 3 - 4 */
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

a : [a] ++ [a] ++ [a]
a : [a]
[a]

(e) `foldr (const l)`
int → [int] → int

(f) `(,"42")`
a → (a, int)

(g) `(,) "42"`
ill-typed

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] → [a]

(j) `let f x = x in (f 'a', f True)`
ill-typed ← for char & bool

(k) `fmap (fmap (+1)) [Nothing]`

[[int]] → [[int]]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x && x]`

(c) `a -> Maybe b`
`\x -> if x then Just 1 else Nothing` a→bool b→int

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith func ilist blist where func i c = if i==1 && c=='c' then true else false`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x y z -> case (x,z) of (Just a, Just b) -> Just (x a b) _ _ -> nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\x y z -> map x z`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x f -> case x of Just a -> map (\x -> (a,x)) (f a) Nothing -> []`

(h) `Eq a => a -> [a] -> [a]`
`intersperse`

(i) `Show a => [a] -> IO String`
`(print (head a))`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`0Func = Ifunc where`
`Ifunc (x,y) = if x == 2 && y == "yes" then true else false`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = (m-m)*m*m  && m >= 0                          }}
  y := 0;
{{ y = (m-m)*m*m  && m >= 0                          }}
  var x := m * m;
{{ y = (m-m)*x    && m >= 0                          }}
  var z := m;
{{ y = (m-z)*x    && z >= 0                          }}
  while (z > 0) {
{{ y+x = (m-z-1)*x && z >= 0                         }} ->>
    z := z - 1;
{{ y+x = (m-z)*x  && z >= 0                          }}
    y := y + x;
{{ y = (m-z)*x    && z >= 0                          }}
  }
{{ y = (m-z)*x    && z >= 0 && !(z > 0)              }} ->>
{{ y = m * m * m }}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (h1:t1) (h2:t2) = h1:h2 ++ (weave t1 t2)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset list = foldr func [[]] list where
  func el acc = map (\x -> el:x) acc

/* [] -> [[]]
   [1] -> [[1],[]]
   [1,2] -> [[1,2],[2],[1],[]] */
```

Snehal Tamsh
11709604

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

[1,2,3,4,5]

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i := a.Length-1
  while (i>0){
    out := out - a[i];
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var out := 0;
  var i := 0;
  while (i < a.Length){
    out := out - a[i];
    i := i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → a → (a, a)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> (x : l)++(l)++([x])`
x : l a ⇒ [a] → [a]
a → [a] → [a]

(e) `foldr (const l)`
a → b → a
ill-typed

(f) `(."42")`
(Empty, String)

(g) `(.) "42"`
a → b → (a, b)
String → b → (String, b)

(h) `reverse . reverse`
[a] → [a]
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
fmap : (a → b) → f a → f(b)
Ill-typed

Sneha1 Tamt
11709804

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
f True = [True]
f False = [False]

(c) `a -> Maybe b`
Nothing

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
let f x y = True in zipWith f [1] ['a']

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
f a b = Just a Just b → Just (a,b)

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
undefined

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
f a = Just a → [a,b] Nothing → [(Nothing, Some 2)]

(h) `Eq a => a -> [a] -> [a]`
undef

(i) `Show a => [a] -> IO String`
undef

(j) `(a,b) -> (a -> b -> c) -> c`
undefined

(k) `((a,b) -> c) -> c`
unde

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                    }}
  y := 0;
{{                                                    }}
  var x := m * m;
{{                                                    }}
  var z := m;
{{                                                    }}
  while (z > 0)
{{                                                    }} ->>
{{                                                    }}
    z := z - 1;
{{                                                    }}
    y := y + x;
{{                                                    }}
}
{{                                                    }} ->>
{{ y = m * m * m}}
```

! a → [a] → [a]

Sneha I Tamt
11709 6064

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

weave [] [] = []

weave (x:xs) (y:ys) = ([x,y]) ++ (weave xs ys)

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

Powerset [] = [[]]

Powerset x:[] = [[x]]

Powerset (x:xs) = [[]] [x] ++ ... ++ Powerset (xs) ++

Powerset (x:(tail xs)) ++ ...

Powerset (x:((head xs):[])) ++ (x:xs)

Powerset [1] = [[1],[]]

Powerset [1,2] = [[1,2],[1],[2],[]]

Jason Liu 118049663

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i := 0;
  while (i < a.Length) {
    out := a[i] - out;
    i := i + 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var i := a.Length - 1;
  while (i >= 0) {
    out := a[i] - out
    i := i - 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const l)` ← Int -> a -> Int
Int -> [Int] -> Int

(f) `(,"42")`
a -> (a, String)

(g) `(,) "42"`
a -> (String, a)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
Eq a => [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(String, Bool).

(k) `fmap (fmap (+1)) [Nothing]`
+1 :: Int -> Int
[Maybe a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [not x]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f i c -> (zipWith f i c)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> listToMaybe ( zipWith (maybeToList a) (maybeToList b))`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f g as -> if g (f (head as)) then [f (head as)] else (map f as)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\m f -> case m of [Just a -> [(a, head (f a))]`
`[Nothing -> []`

(h) `Eq a => a -> [a] -> [a]`
`\x as -> if x == x then x:as else as`

(i) `Show a => [a] -> IO String`
`\x -> putStrLn (show (head x))`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{    0 = m*m*(m-m)                                          }}
  y := 0;
{{    y = m*m*(m-m)                                          }}
  var x := m * m;
{{    y = x*(m-m)                                            }}
  var z := m;
{{    y = x*(m-z)                                            }}
  while (z > 0) {
{{    y = x*(m-z)                          &&   z>0          }} ->>
{{    y+x = x*(m-z-1)                                        }}
    z := z - 1;
{{    y+x = x*(m-z)                                          }}
    y := y + x;
{{    y = x*(m-z)                                            }}
}
{{    y = x*(m-z)                          &&   !(z>0)       }} ->>
{{ y = m * m  * m}}
```

z	y
m	0
m-1	m*m*1
m-2	m*m*2
m-3	m*m*3
2	m*m*(m-
1	m*m*
0	m* m*
	m* m* m

m*m*(m-2)

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) =
    x:y: weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

1, 2, 3, 4

```
powerset [] = []
powerset [x] = [[x]]
powerset [x,y] = [[x,y],[x],[y]]
powerset lst = map powerset (tails lst)
```

Nela Trakic 117896541

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

end to start

start to end

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length-1;
  out := 0;
  while (i >= 0)
    out := a[i]-out;
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length)
    out := a[i]-out;
    i := i+1
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [[a]]

(e) `foldr (const 1)`
int → [int] → int

(f) `(,"42")`
a → (a, string)

(g) `(,) "42"`
a → b → (a, string)

(h) `reverse . reverse`
[a] → [a] → [a]

(i) `\l -> filter (<42) (1 ++ l)`
[int] → (a → Bool) → [int] → [int]

(j) `let f x = x in (f 'a', f True)`
(a → b) → (b, b) → b

(k) `fmap (fmap (+1)) [Nothing]`
(int → b) → f Maybe a → f b

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`foo t = [t, True]`

(c) `a -> Maybe b`
`foo a = Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`foo f i c = [True, f i c]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`foo f a b = f (Just a) (Just b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`foo f1 f2 ls = if f2 (f1 (ls !! 1)) == True then [f1 (ls !! 1)] else [f1 (ls !! 0)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\a b -> zip (a) (b a)`

(h) `Eq a => a -> [a] -> [a]`
`\n l -> delete n l`

(i) `Show a => [a] -> IO String`
`foo a = show (putStrLn (a ++ []))`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`foo f = f (5, True)`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m >=0 && 0 == m*m*m                    }}
  y := 0;
{{ m >=0 && y == m*m*m                    }}
  var x := m * m;
{{ m >=0 && y == m*x                      }}
  var z := m;
{{ z >=0 && y == z*x                      }}
  while (z > 0) {
{{ z >=0 && y == z*x && z>0               }} ->>
{{ (z-1) >=0 && (y+x) == (z-1)*x          }}
    z := z - 1;
{{ z >=0 && (y+x) == z*x                  }}
    y := y + x;
{{ z >=0 && y == z*x                      }}
  }
{{ z >=0 && y == z*x && !(z>0)            }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] _ = []
weave _ [] = []
weave (x:xs) (x2:xs2) = x : x2 : weave xs xs2
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [a] -> [[a]]
powerset [] = []
powerset (x:xs)
    | [x] = [x] ++ powerset xs
    | otherwise = [x : powerset xs] ++ powerset xs
```

Pete Smith
UID 116 650 786

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var out := 0;
  if (a.Length >0) {
    var i := 0;
    out = a[i];
    i := i + 1;
    while (i < a.Length) {
      out := out - a[i];
      i := i + 1;
    }
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var out := 0;
  if (a.Length > 0) {
    var i := a.Length - 1;
    out = a[i];
    i := i - 1;
    while (i >= 0) {
      out := out - a[i];
      i := i - 1;
    }
  }
}
```

const :: a -> b -> a

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)` a -> b -> (

(c) `\x y -> if x == y then show x else show (x,y)` a -> b -> String

(d) `\x l -> x : l ++ l ++ [x]` a -> [a] -> [a]

(e) `foldr (const 1)` b -> b -> a -> b

(f) `(,"42")` b -> (a, b)

(g) `(,) "42"` b -> (a, b)

(h) `reverse . reverse` b -> a -> a -> (

(i) `\l -> filter (< 42) (l ++ l)` [a] -> [a]

(j) `let f x = x in (f 'a', f True)` (a -> b) -> c -> (d, e)

(k) `fmap (fmap (+1)) [Nothing]`

const :: a -> b -> a
foldr :: (a -> (b -> b) -> b -> t a -> b
fmap :: (a -> b) -> f a -> f b
(.) :: (b -> c) -> (a -> b) -> a -> c
(,) :: a -> b -> (a, b)
filter :: (a -> Bool) -> [a] -> [a]
(++) :: [a] -> [a] -> [a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]` `\x -> map (\y -> if y > 0 == x then x else not x) [1, 2, 3]`

(c) `a -> Maybe b` `\x -> if x > 0 then Just "a" else (if x == 0 then Just "b" else Nothing)`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]` `\f g xs -> map (g (f)) xs`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]` `\x f -> map (\y -> (x,y)) (f x)`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String` `\f xs -> map f xs`

(j) `(a,b) -> (a -> b -> c) -> c` `\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 + m = m                                                    }}
  y := 0;
{{ 0 + y = 0 && 0 + m = m                                       }}
  var x := m * m;
{{ 0 + y = y && 0 + x = m * m && 0 + m = m                      }}
  var z := m;
{{ 0 + y = y && 0 + x = x && 0 + z = m                          }}
  while (z > 0) {
{{ 0 + y = y && 0 + z = z && (z > 0)                            }} ->>
{{ 0 + y + x = y + x && 0 + z - 1 = z - 1 && (z - 1 > 0)        }}
    z := z - 1;
{{ 0 + y + x = y + x && 0 + z = z && (z >= 0)                   }}
    y := y + x;
{{ 0 + y = y && 0 + z = z && (z >= 0)                           }}
  }
{{ 0 + y = y && 0 + z = z && !(z > 0)                           }} ->>
{{ y = m * m * m}}
```

I forgot to put "==" instead of just "=" but it will be very messy if I redo all of it

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []

weave (x:xs) (y:ys) = (x:(y:(weave xs ys)))
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset xs = pAux xs 0 (length x) where
  pAux xs i n = if i < n then
                  (xs !! i) : (pAux xs (i+1) n)
                else
                  []
```

Typeclass Definitions

```
class Semigroup a where
  (<>)   :: a -> a -> a

class Semigroup m => Monoid m where
  mempty :: m

class  Show a  where
  show   :: a   -> String

class  Eq a  where
  (==)   :: a -> a -> Bool
  (/=)   :: a -> a -> Bool

class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min             :: a -> a -> a

class Functor f where
  fmap   :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
  pure   :: a -> f a
  (<*>)  :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b

class Foldable t where
  foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
  arbitrary :: Gen a
  shrink    :: a -> [a]

class Num a where
  (+), (-), (*)       :: a -> a -> a
  negate              :: a -> a
  abs                 :: a -> a
  signum              :: a -> a
  fromInteger         :: Integer -> a
```

Prelude Functions and Datatypes

Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not  :: Bool -> Bool
```

Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null   :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl  :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr  :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or  :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*>) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence  :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...
```

1 Hoare Rules

```
  { P } s1 { Q }
  { Q } s2 { R }
-------------------  (hoare_seq)
{ P } s1; s2 { R }


------------------------ (hoare_asgn)
{Q [x := a]} x := a {Q}


  {P'} c {Q'}
P ==> P'    Q' ==> Q
--------------------- (hoare_consequence)
   {P} c {Q}

        {P &&  b} s1 {Q}
        {P && !b} s2 {Q}
-------------------------------- (hoare_if)
{P} if b { s1 } else { s2 } {Q}

        {P && b} s {P}
---------------------------- (hoare_while)
{P} while b { s } {P && !b}
```

RAGHAV AGGARWAL
11790106

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0
  while i >= 0
    invariant i > 0
  {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while i < a.Length
    invariant i <= a.Length
  {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`

a → a → IO String

(d) `\x l -> x : l ++ l ++ [x]`

a → [a] → [a]

(e) `foldr (const 1)`

b → [Int] → b

(f) `(.)"42"`

ill-typed

(g) `(.) "42"`

String → (String, String)

(h) `reverse . reverse`

a → a

(i) `\l -> filter (< 42) (1 ++ l)`

[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`

ill-typed

(k) `fmap (fmap (+1)) [Nothing]`

ill-typed

x : [x]

RAGHAV AGARWAL

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x && x]`

(c) `a -> Maybe b`
`\x -> Maybe x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f a c -> if (head a == 5 && head c == 'c') then [f (head a) (head c)] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f x y -> match (x, y, z) of`
`| (Maybe a, Maybe b) -> f a b ;`
`| (_,_) -> f a b ;`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`
`(i) \t f -> match t of`
`| (a,b) -> f a b ;`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`
`(k) f (x, y) ->`

:) `\f g l -> match l of`
`| [x] -> [f x]`
`| _ -> if (g (f a)) then [] else [f x] ;`

) `\x f -> match x of`
`| Maybe a -> [a, head (f a)]`
`| Nothing -> []`

`\x l -> if x < x then x : l else l`

) `\x -> show (head x)`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = (m-m)·m²  ∧  m ≥ 0                          }}
  y := 0;
{{ y = (m-m)·m²  ∧  m ≥ 0                          }}
  var x := m * m;
{{ y = (m-m)·m²  ∧  m ≥ 0                          }}
  var z := m;
{{ y = (m-z)·m²  ∧  z ≥ 0                          }}
  while (z > 0) {
{{ y = (m-z)·m²  ∧  z ≥ 0  ∧  z > 0                }} ->>
{{ y+x = (m-z+1)·m²  ∧  z-1 ≥ 0                    }}
    z := z - 1;
{{ y+x = (m-z)·m²  ∧  z ≥ 0                        }}
    y := y + x;
{{ y = (m-z)·m²  ∧  z ≥ 0                          }}
}
{{ y = (m-z)·m²  ∧  z ≥ 0  ∧  z ≤ 0                }} ->>
{{ y = m * m  * m}}
```

RAGHAV AGGARWAL

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) =
    x : y : (weave xs ys)
```

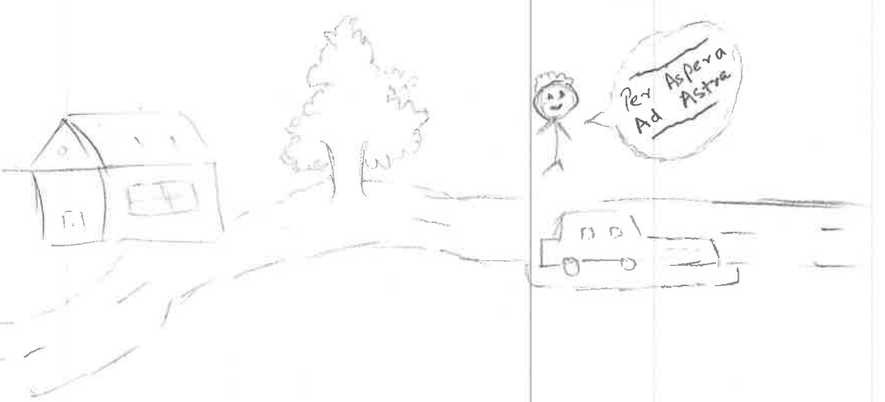
(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset l = helper l [] 0

helper l sub (l.length-1) = [sub]

helper (x:xs) sub i =
    (helper xs (x:sub) (i+1))
    ++
    (helper xs sub (i+1))
```


Per Aspera
Ad Astra

REBECCA BEYENE
118498775

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {

if a.length == 0 {
    out := 0
} else {
    out := out - f1(a[1..])
}

}
```

```
method f2(a:array<int>) returns (out:int) {

if a.length == 0 {
    out := 0
} else {
    var i := a.length - 1
    while (i ≥ 0) {
        out := f2(a[0..i-1]) - a[i]
    }
}

}
```

[1, 2, 3]
1 - (2 - (3 - (0)))

((1-2)-3)-4)

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`[a] -> a`

(b) `\x y -> (x,y)`
a -> b -> (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const 1)`
[a] -> [a] -> a

(f) `(."42")`
a -> (a, String)

(g) `(.) "42"`
String -> String

(h) `reverse . reverse`
[a] -> [a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
(a -> b) -> b -> Bool

(k) `fmap (fmap (+1)) [Nothing]`
-> [Maybe Int]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\a -> if a then [a] else []`

(c) `a -> Maybe b`
`\a -> if a == Empty then Nothing else`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`map (\a b -> a == a to i (b)) [a] [b]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> if (a == Nothing || b == Nothing) then Nothing else (f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m*m*0                                          }}
  y := 0;
{{ y = m*m*0                                          }}
  var x := m * m;
{{ y = m*x*0                                          }}
  var z := m;
{{ y = m*x*z                                          }}
  while (z > 0) {
{{ (z > 0) && (y == m*x*z)                            }} ->>
{{ (y+x) == m*x*(z+1)                                 }}
    z := z - 1;
{{ y+x == m*x*(z+1)                                   }}
    y := y + x;
{{ y == m*x*(z+1)                                     }}
  }
{{ !(z > 0) && y == m*x*(z+1)                         }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave a b = zipwith(a,b)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset a = zipwith (map(:))
```

Can use recurse

Daniel Lam

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = fcldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while i >= 0 {
    out := a[i] - out;
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
a → [a] → a

(f) `(."42")`
a → (String, a)

(g) `(.) "42"`
a → (String, a)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] → [a]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
Maybe Int

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> x : [True]`

(c) `a -> Maybe b`
undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f i:is c:cs -> if i == 1 && c == 'a' then (f i c) : [False] else [True]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f (Maybe a) (Maybe b) -> Maybe (f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f g a:as -> if g(f(a)) == True then [f(a)] else [f(a)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\(Maybe a) f -> f`

(h) `Eq a => a -> [a] -> [a]`
`\y (x:xs) -> if x == y then xs else xs`

(i) `Show a => [a] -> IO String`
`\(a:as) -> show a`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

y + z·x = m·m·m

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules: write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m >=0 && 0+m*m*m = m*m*m                                  }}
  y := 0;
{{ m >=0 && y+m*m*m = m*m*m                                  }}
  var x := m * m;
{{ m >=0 && y+m*x = m*m*m                                    }}
  var z := m;
{{ z >=0 && y+z*x = m*m*m                                    }}
  while (z > 0) {
{{ z >=0 && y+z*x = m*m*m && z>0                             }} ->>
{{ z-1 >=0 && y+x+(z-1)*x = m*m*m                            }}
    z := z - 1;
{{ z >=0 && y+x+z*x = m*m*m                                  }}
    y := y + x;
{{ z >=0 && y+z*x = m*m*m                                    }}
  }
{{ z>=0 && y+z*x=m*m*m && !(z>0)                             }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) = x : y : (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset x:xs = inits (x:xs) ++ powerset (xs)
```

Samuel Badalov 118828176

CMSC 433: Midterm Exam (Spring 2024)

foldl 1-2-3-4 -10-5

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
var i:int := a.length-1
out:=0
while (i >= 0) {
    out := out - a[i]
    i := i-1
}
}
```

```
method f2(a:array<int>) returns (out:int) {
var i:int := 0
out:=0
while (i <= a.length-1) {
    out := out - a[i]
    i := i+1
}
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const l)`
[a] -> Int

(f) `(,"42")`
(, [Char])

(g) `(.) "42"`
ill-typed

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

f :: [Int] -> Char -> Bool

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

```
foo a = if a then [a] else [a]
```

(c) `a -> Maybe b`

```
foo a = Just a
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
foo f a b = let bar = ['a'] ++ b in [f x (head a) (head b) == True]
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
foo f a b = match a,b | Maybe x,Maybe y -> f x y
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
foo f g a = if g $ f $ head a then f $ head a else f $ head a
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
foo a f = match a | Maybe x -> [(x, head $ f a)]
```

(h) `Eq a => a -> [a] -> [a]`

```
foo a ls = [a] ++ nub ls
```

(i) `Show a => [a] -> IO String`

```
foo a = putStrLn $ show $ head a
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
foo f g = g f g b
```

(k) `((a,b) -> c) -> c`

```
foundefined
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

y	x	z
0	m^2	m
m^2	m^2	m-1
$2m^2$	m^2	m-2

$y = (m-z) \cdot m \cdot m$

$z <= 0$

$y = m-z \cdot m \cdot m$

-> y becomes m^2

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0=(m-m) (m*m) && m >=0                    }}
  y := 0;
{{ y= (m-m) (m*m)  && m >=0                  }}
  var x := m * m;
{{ y=(m-m)*x  && m >=0                       }}
  var z := m;
{{ y=(m-z) x  && z >=0                       }}
  while (Z > 0) {
{{ y= (m-z)*x  && z >=0                      }} ->>
{{ y+x=(m-(z-1))  && z-1 >=0                 }}
    z := z - 1;
{{ y+x=(m-z)*x  && z >=0                     }}
    y := y + x;
{{ y=(m-z)*x  && z >=0                       }}
}
{{ y=(m-z)*x && !(z>0) && x=m*m && z>=0      }} ->>
{{ y = m * m  * m}}
```

z <= 0

init [1,2,3] → [1,2] → [1] → []

tails [1,2,3] → [1,2,3] → [2,3] → [3]

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

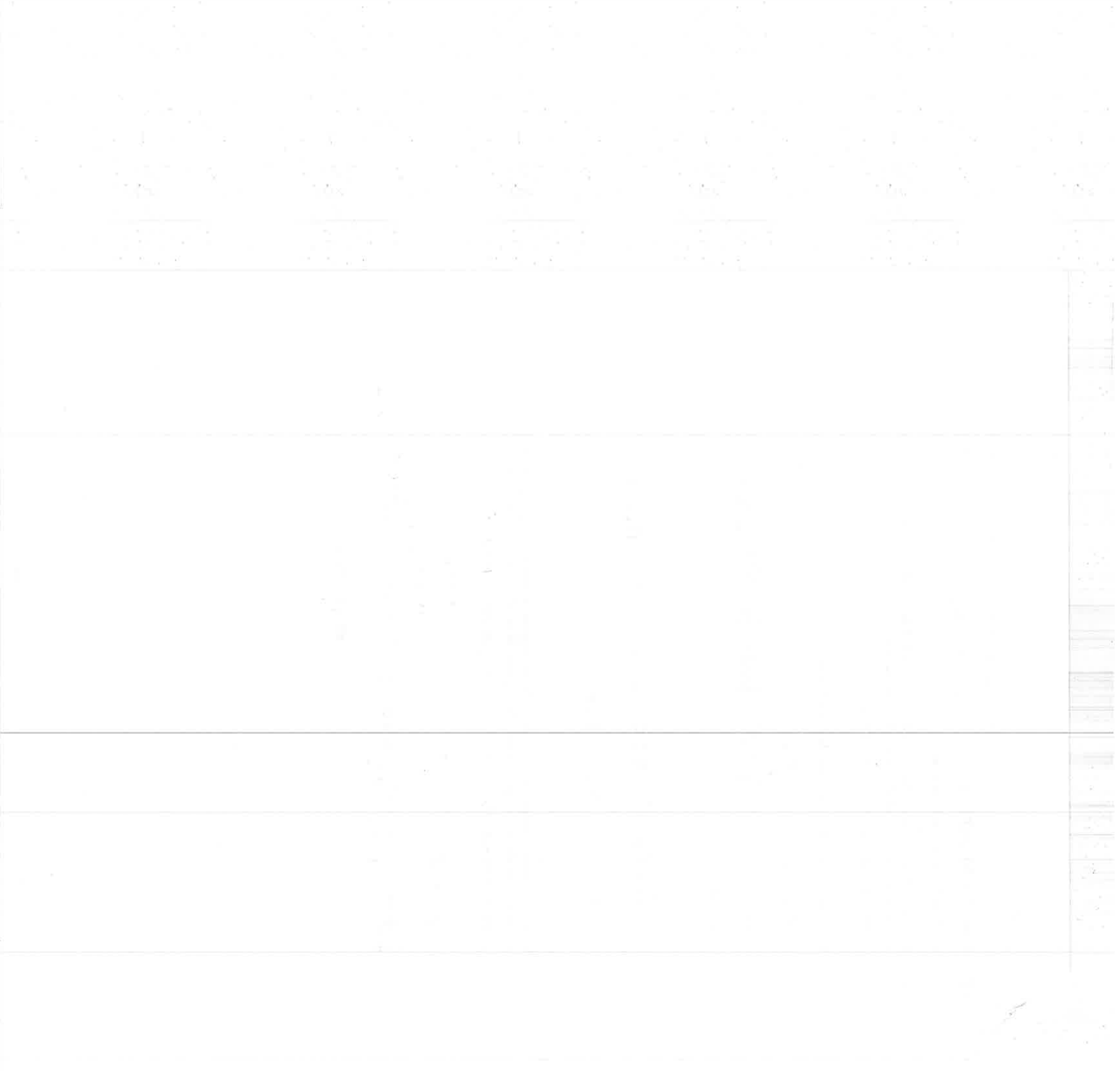
You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) = foldr foo [] zip (x:xs) (y:ys)
  where
    foo (a,b) acc = a : (b : acc)
    foo _ acc = acc
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset (x:xs) = nub $ (init (x:xs)) ++ (tails (x:xs))
                        ++ (powerset xs)
```



Kobe Wang 11801356S

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

func -> acc -> list

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    out:= 0;
    var len:= a.Length-1;
    while(len >=0){
        out := out - a[len];
        len := len - 1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    out:=0;
    var i:=0;
    while(i < a.Length){
        out := out - a[i];
        i:= i+1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> b -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> b ->

(e) `foldr (const 1)`
b -> [a] -> b

(f) `(,"42")`
(a, b)

(g) `(,) "42"`
b -> (a, b)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (1 ++ l)`
a -> (a -> bool) -> [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
ill typed

(k) `fmap (fmap (+1)) [Nothing]`
[a] -> [a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [True] else [False]`

(c) `a -> Maybe b`
`\x -> if x == x then Nothing else x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`foo f lst lst2 if f head lst == 1 && head lst2 == 'c' then [True] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`let f x y = if x == Nothing && y == Nothing then Nothing else`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`let f func func2 lst = if (func2 (func head lst)) then tail lst`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`let f a func = zip (func a)`

(h) `Eq a => a -> [a] -> [a]`
`lookup zip a`

(i) `Show a => [a] -> IO String`
`if x =`

(j) `(a,b) -> (a -> b -> c) -> c`
`let (x,y) f = f(x,y)`

(k) `((a,b) -> c) -> c`
`let f`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                          }}
  y := 0;
{{                                          }}
  var x := m * m;
{{                                          }}
  var z := m;
{{                                          }}
  while (z > 0) {
{{                                          }} ->>
{{                                          }}
    z := z - 1;
{{                                          }}
    y := y + x;
{{                                          }}
  }
{{                                          }} ->>
{{ y == m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

fold

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave [x:xs] [] = [x] : weave xs []
weave [] [y:ys] [y] : weave ys []
weave [x:xs] [y:ys] :
```

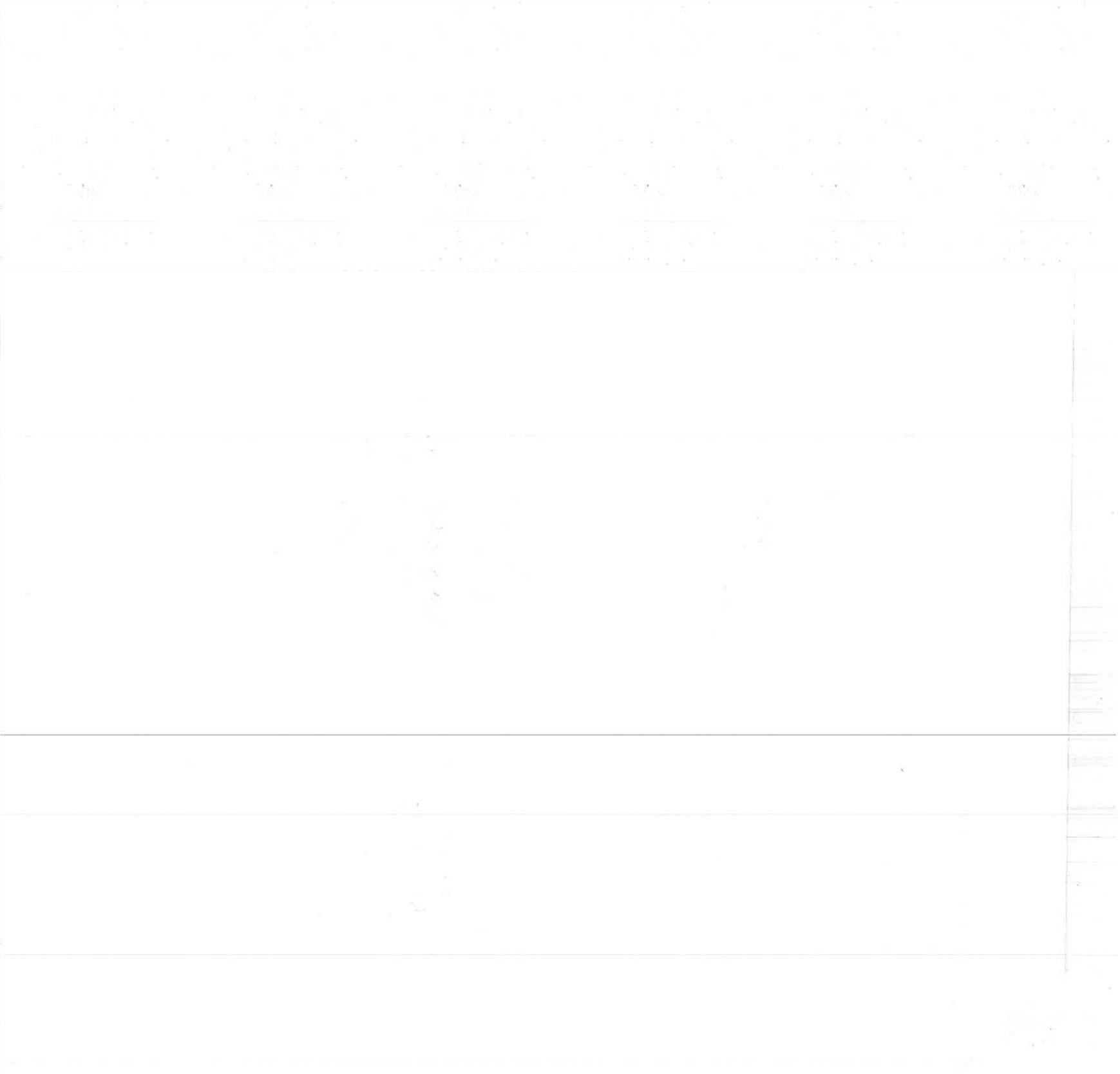
count

```
if count `mod` 2 == 0
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
var set := {} in
powerset lst = fold (x -> if lookup x set then x) [] lst
```



Shreyaj Bhandari
117526629

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i : int := 0;
    out := 0;
    while i < a.length {
        i := i + 1
        out := out - a[i-1]
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i : int := 0;
    out := 0;
    while i < a.length {
        i := i + 1
        out := out - a[i-1]
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
Foldable t ⇒ b ⇒ ta → b

(f) `(.) "42"`
ill-typed

(g) `(.) "42"`

(h) `reverse . reverse`

(i) `\l -> filter (< 42) (l ++ l)`

(j) `let f x = x in (f 'a', f True)`
a → a

(k) `fmap (fmap (+1)) [Nothing]`
[

const :: a → b → a
foldr (const 1) (a → b ⇒ b)
foldr :: Foldable t ⇒ b → ta → b

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then x:[] else x:[]`

(c) `a -> Maybe b`
`\a -> if true then Just a else Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f a b -> if a==[2] && b==['c'] && f a b then [f a b] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f Just a Just b -> Just (f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 (x:[]) -> if f2 (f1 x) then [f1 x] else [f1 x]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\some a f -> if f a == (x:[]) then [(a,x)] else [(a,x)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) -> f -> f a b`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with –>>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0+(m*m)==m*m*m && (m-0)>=0 }}
  y := 0;
{{ y+(m*m)==m*m*m && (m-0)>=0 }}
  var x := m * m;
{{ y+x==m*m*m && (m-0)>=0 }}
  var z := m;
{{ y+x==m*m*m && (z-0)>=0 }}
  while (z > 0) {
{{ y+x==m*m*m && (z-0)>=0 }} ->>
{{ y+x==m*m*m && (z-1)>=0 }}
    z := z - 1;
{{ y+x==m*m*m && (z)>=0 }}
    y := y + x;
{{ y==m*m*m && (z)>=0 }}
}
{{ y==m*m*m && !(z!=0) }} ->>
{{ y = m * m  * m}}
```

y=y+x

foldr

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [Int] -> [Int] -> [Int]
weave [] [] = []
weave (x:xs) (y:ys) = x:y: weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [int] -> [[Int]]
powerset [] = [[]]
powerset (x:xs) = (x:xs):[x]:xs: foldr (\i acc -> [x:i]:acc) xs []:
                                                    powerset xs
```

CMSC 433: Midterm Exam (Spring 2024)

Hayden Miller

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    out := 0;
    var i := a.Length;
    while (i >= 0) {
        out := a[i] - out;
        i := i - 1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    out := 0;
    var i := 0;
    while (i < a.Length) {
        out := out - a[i];
        i := i + 1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const l)`
b -> [a] -> b

(f) `(,"42")`
Ill-typed

(g) `(,) "42"`
(String,)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
[Maybe Int]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

```
\x -> [x && True]
```

(c) `a -> Maybe b`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

```
\x xs -> map (\y -> x == y) xs
```

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

```
\(a,b) f -> f a b
```

(k) `((a,b) -> c) -> c`

```
d
func where
func f (a:as) (b:bs) = (f a b) : (func f as bs)
func f _ _ = []

e
func where
func f (Just a) (Just b) = Just (f a b)
func f _ _ = Nothing

f
func where
func f g (a:as) = if g b then b:(func f g as)
                  else [] where b = f a
func f g [] = []
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules: write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m*m*(m-m) && m*m = m*m                    }}
  y := 0;
{{ y = m*m*(m-m) && m*m = m*m                    }}
  var x := m * m;
{{ y = x*(m-m) && x = m*m                        }}
  var z := m;
{{ y = x*(m-z) && x = m*m                        }}
  while (z > 0) {
{{ z>0 && y = x*(m-z) && x = m*m                 }} ->>
{{ y+x = x*(m-(z-1)) && z-1 >= 0 && x = m*m      }}
    z := z - 1;
{{ y+x = x*(m-z) && z >= 0 && x = m*m            }}
    y := y + x;
{{ y = x*(m-z) && z >= 0 && x = m*m              }}
  }
{{ y = x*(m-z) && z >= 0 && z <= 0 && x = m*m    }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function **weave** that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave (x:xs) ys = x:(weave ys xs)
weave [] ys = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset = foldr f [] where
  f x [] = []
  f x ys = (map (\y -> x:y) ys) ++ ys
```

Jai Raghavan

CMSC 433: Midterm Exam (Spring 2024)

0 - 1 = -1
-1 - 2 = -3
-3 - 3 = -6
-6 - 4 = -10

4 - 0 = 4 1 - 3 = -2
3 - 4 = -1
2 - (-1) = 3

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

[1,2,3,4] -> -2

[1,2,3,4] ->

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i = a.Length - 1;
  while i >= 0 {
    out := a[i] - out;
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var i := 0
  while i < a.Length {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a - b -> (a , b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const 1)`
ill-typed

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
ill-typed

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> (Int -> Bool) -> [Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [false]`

(c) `a -> Maybe b`
`\x -> Just [x]`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith (\(x:xs) (y:ys) -> x == 1 && y == 'c') [1,2] "hi"`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> if isNothing a && isNothing b then f a b else f a b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\`

(h) `Eq a => a -> [a] -> [a]`
`delete`

(i) `Show a => [a] -> IO String`
`\(x:xs) -> show x`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = 0 * m * m }}
  y := 0;
{{ y = 0 * m * m }}
  var x := m * m;
{{ y = 0 * x }}
  var z := m;
{{ y = (m-z) * x }}
  while (z > 0) {
{{ z > 0 && z >= 0 && y = (m-z) * x }}
{{ z-1 >= 0 && y+x = (m-(z-1)) * x }}
    z := z - 1;
{{ z >= 0 && y+x = (m-z) * x }}
    y := y + x;
{{ z >= 0 && y = (m-z) * x }}
  }
{{ !(z>0) && z >= 0 && y = (m-z) * x }}
{{ y = m * m * m}}
```

}} ->>
}}
}}
}}
}} ->>
}}
}}
}}
}} ->>

y = x * z

y = m * m * z

y = (m-z) * x

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

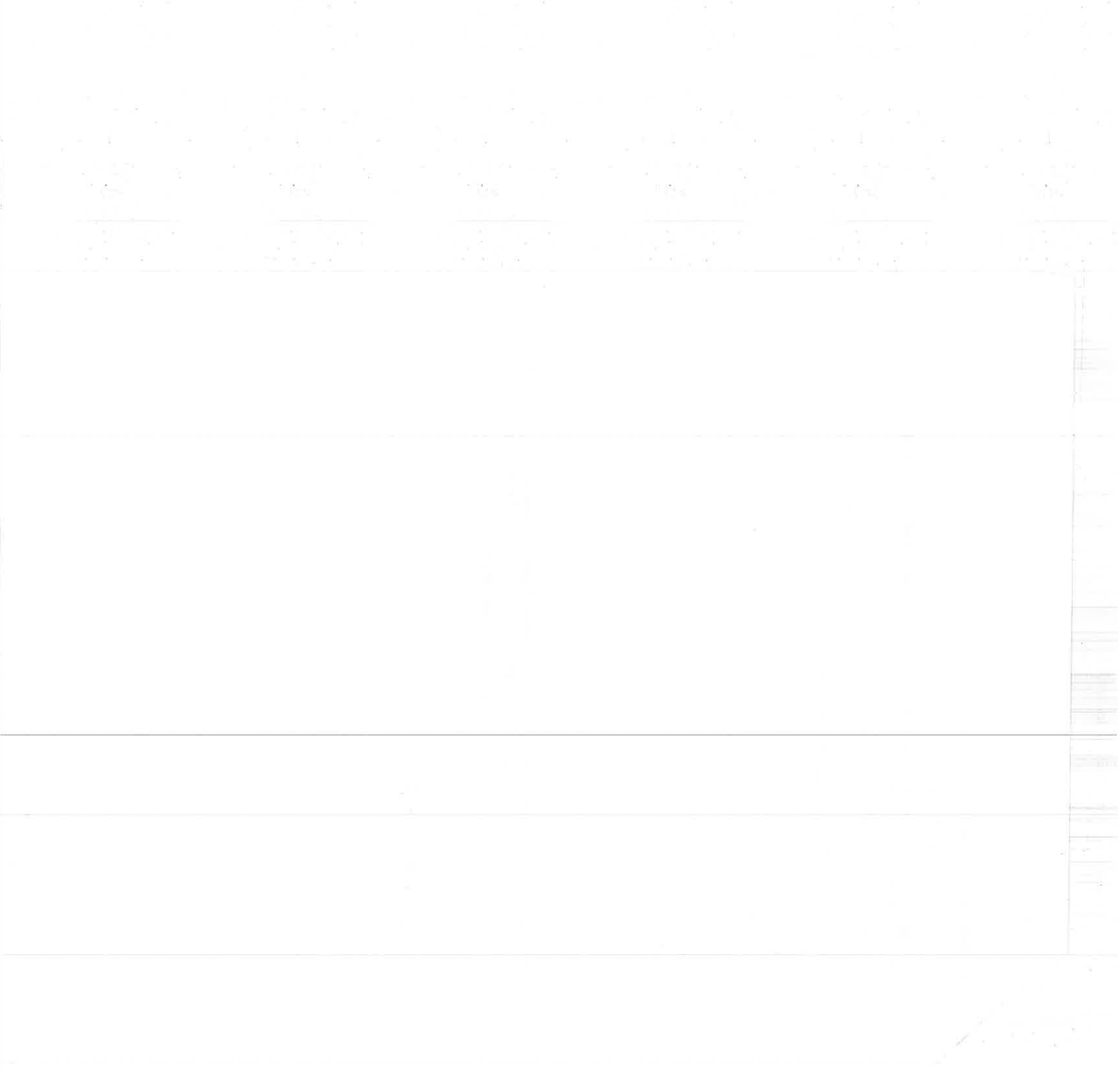
```
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = x : y : (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]

powerset (x:xs) = (x:xs) : x : powerset xs
```



Andy Wong

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

(Int → b → Int)
Int

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
Int → [Int] → Int

(f) `(,"42")`
ill typed

(g) `(,) "42"`
String → b → (String, b)

(h) `reverse . reverse`
[a] → [a] → [a]

(i) `\l -> filter (< 42) (1 ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\b -> [b]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f is cs -> zipWith f is cs`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> maybe b (maybe a f (Nothing) Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 l -> undefined`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\a f ->`

(h) `Eq a => a -> [a] -> [a]`
`Eq a = undefined`

(i) `Show a => [a] -> IO String`
`Show a = undefined`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`\f -> undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m	x	z	y
3	9	3	0
	9	2	9
	9	1	18
	9	0	27

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules --write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m >= 0 && 0 = (m*m)(m-m) && m*m = m*m }}
  y := 0;
{{ m >= 0 && y = (m*m)(m-m) && m*m = m*m }}
  var x := m * m;
{{ m >= 0 && y = x(m-m) && x = m*m }}
  var z := m;
{{ z >= 0 && y = x(m-z) && x = m*m }}
  while (z > 0) {
{{ z >= 0 && y = x*(m-z) && x = m*m && z > 0 }} ->>
{{ z-1 >= 0 && y = x(m-(z-1)) && x = m*m }}
    z := z - 1;
{{ z >= 0 && y+x = x(m-z) && x = m*m }}
    y := y + x;
{{ z >= 0 && y = x(m-z) && x = m*m }}
  }
{{ z >= 0 && y = x(m-z) && x = m*m && !(z > 0) }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

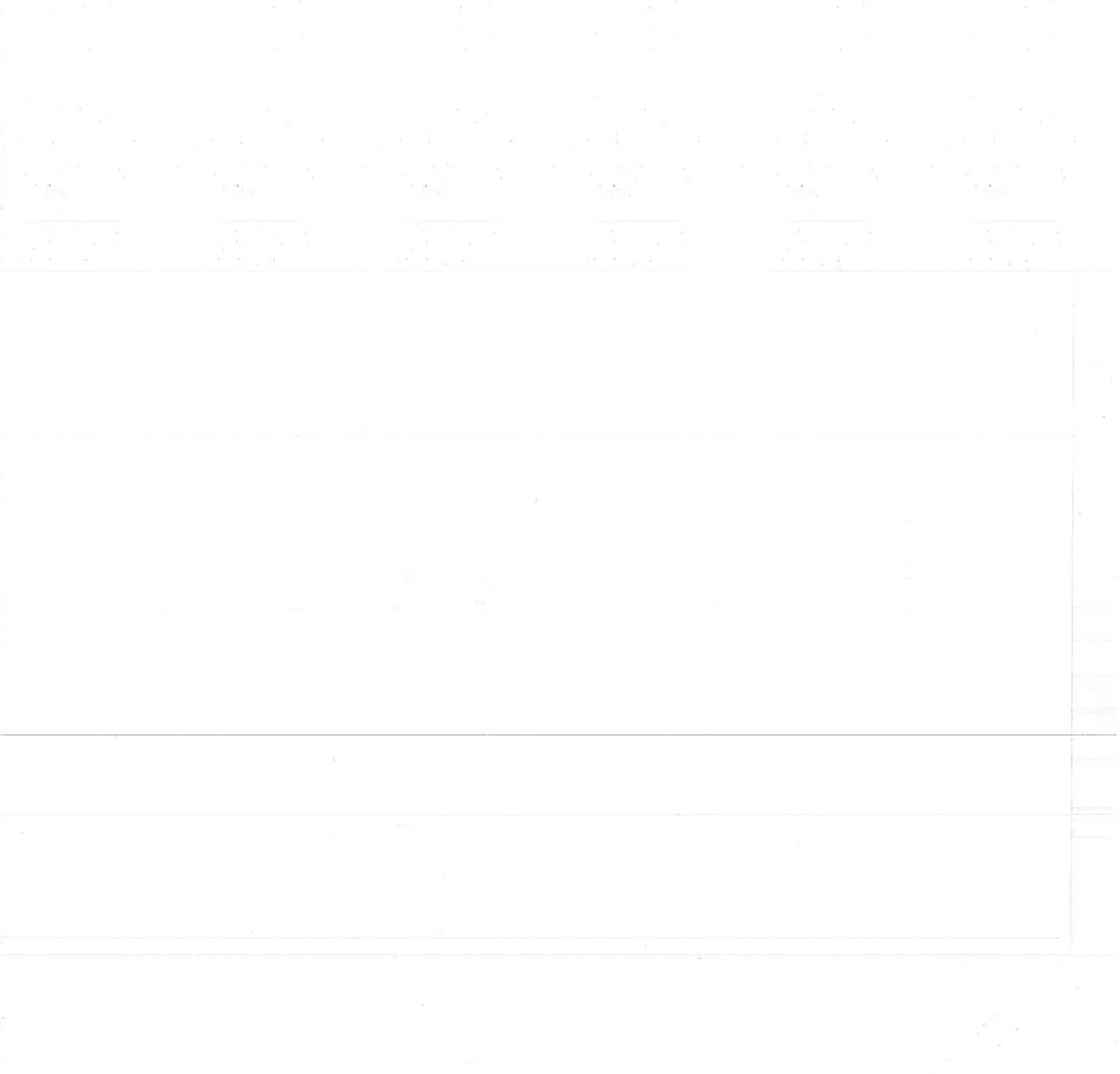
You can assume that the lists have the same length.

```
weave [] l2 = l2
weave (x:xs) l2 = x : (weave2 xs l1)
    where
      weave2 l1 (y:ys) = y : (weave l1 ys)
      weave2 l1 [y] = [y]
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset (x:xs) = (join' x xs) ++ (powerset xs)
    where
      join' x [] = [x]
      join' x (y:ys) = [x,y] ++ (join' x ys)
```



Brandon Switzer

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

(,) :: a → b → (a, b)

const : a → b → a

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
ill-typed

(f) `(,"42")`
b → (b, String)

(g) `(,) "42"`
b → (String, b)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
Num a => [a]

fmap (+1) Num a → Num a
fmap (f Int → f Int) f f Int

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [not x]`

(c) `a -> Maybe b`
`undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f i c -> zipWith f i c`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f (Just x) (Just y) -> Just (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\fa fb (x:xs) -> [fa x]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\(Just x) f -> zip [x] (f x)`

(h) `Eq a => a -> [a] -> [a]`
`\x y -> x : y`

(i) `Show a => [a] -> IO String`
`\x:xs -> putStrLn (show x)`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

z >= 0 && y == (m-z) * x

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m>=0 && 0==(m-m)*m*m                    }}
  y := 0;
{{ m>=0 && y==(m-m)*m*m                    }}
  var x := m * m;
{{ m>=0 && y==(m-m)*x                      }}
  var z := m;
{{ z>=0 && y==(m-z)*x                      }}
  while (z > 0) {
{{ z>=0 && y==(m-z)*x && z>0                }} ->>
{{ z-1>=0 && y+x==(m-z-1)*x                }}
    z := z - 1;
{{ z>=0 && y+x==(m-z)*x                    }}
    y := y + x;
{{ z>=0 && y==(m-z)*x                      }}
}
{{ z>=0 && y==(m-z)*x && !(z>0)            }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave x:xs y:ys = x : y : (weave xs ys)
weave [] y:ys = []
weave x:xs [] = []
weave [] [] = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset x:xs = (powerset xs) ++ (map ((:) x) (powerset xs))
```

Daniel Truong

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  i = a.length - 1
  length = a.length
  while (i < a.length)
  if a.length >= 2 then {
    out = a[i]
    while (i >= 0)
    { out = out - a[i-1]
      i -= 1
    }
  else if a.length == 1 return a[0]
  else if a.length == 0 return 0
  return out
}
```

```
method f2(a:array<int>) returns (out:int) {
  i = 0;
  length = a.length - 1;
  if a.length >= 2 { out = a[0];
  while (i < length)
  {
    out = out - a[i+1];
    i += 1;
  }
  }elif a.length == 1 {return a[0]}
   elif a.length == 0 {return 0}
   return out
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a` $a \to a$

(b) `\x y -> (x,y)`
$a \to b \to (a, b)$

(c) `\x y -> if x == y then show x else show (x,y)`
$a \to b \to \text{String}$

(d) `\x l -> x : l ++ l ++ [x]`
$a \to [a] \to [a]$

(e) `foldr (const 1)`
$a \to t\ a \to a$

(f) `(."42")`

(g) `(.) "42"`

(h) `reverse . reverse`
$[a] \to [a]$

(i) `\l -> filter (< 42) (l ++ l)`
$[a] \to [a]$

(j) `let f x = x in (f 'a', f True)`
$(a \to b) \to$ $\to$

(k) `fmap (fmap (+1)) [Nothing]`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

(c) `a -> Maybe b`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$y = (m^2)(m+m+m)$
$x \quad z$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m·m·m                                  }}
  y := 0;
{{ y = m·m·m                                  }}
  var x := m * m;
{{ y = x·m                                    }}
  var z := m;
{{ y = x·z                                    }}
  while (z > 0) {
{{ z ≥ 0 && y = x·z                           }} ->>
{{ y+x = x·(z-1)                              }}
    z := z - 1;
{{ y+x = x·z                                  }}
    y := y + x;
{{ y = x·z                                    }}
  }
{{ !(z > 0) && y = x·z                        }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = [x,y] : weave (xs) (ys)
weave []     _      = []
weave _      []     = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset x = helper x x
  where
    helper (x:xs) (y:ys) = if x == y
        if x == y then helper x ys
        else zip x y
    helper [] _ = []
    helper _ [] = []
```

Ruonan Feng

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.Length-1;
  var acc:int := 0;
  while i >=0 {
    acc := a[i] - acc;
    i := i-1;
  }

  return acc;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := 0;
  var acc : int := 0;
  while i <= a.Length-1 {
    acc := acc - a[i];
    i := i+1;
  }
  return acc;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`

a → a → String

(d) `\x l -> x : l ++ l ++ [x]`

a → [a] → [a]

(e) `foldr (const l)`

a→b→a
const l
foldr f acc ... → acc
a

ill-typed

(f) `(."42")`

ill-typed

(g) `(,) "42"`

ill-typed

(h) `reverse . reverse`

[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`

[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`

ill-typed

(k) `fmap (fmap (+1)) [Nothing]`

[Maybe Int] → [Maybe Int]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> x : [True]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f (c:cs) (b:bs) -> (f c b) : [True] where f c b = if c=='a' then True else False`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`undefined`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 (a:as) -> if (f2 (f1 a)) then (f1 a):[] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`func (Just a) f = zip [a] (f a)`
`func (Nothing) f = []`

(h) `Eq a => a -> [a] -> [a]`
`\a (x:xs) -> if a==x then a:(x:xs) else xs`

(i) `Show a => [a] -> IO String`
`\(x:xs) -> show x`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m² m-1 m² invariant y = (m - z)*x m-2 m² + m²

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = (m-m)*(m*m) && m>=0 }}
  y := 0;
{{ y = (m-m)*(m*m) && m>=0 }}
  var x := m * m;
{{ y = (m-m)*x && m>=0 }}
  var z := m;
{{ y = (m-z)*x && z>=0 }}
  while (z > 0) {
{{ y = (m-z)*x && z>0 && z>=0 }} ->>
{{ y + x = (m-(z-1))*x && z>=0 }}
    z := z - 1;
{{ y + x = (m-z)*x && z>=0 }}
    y := y + x;
{{ y = (m-z)*x && z>=0 }}
}
{{ y = (m-z)*x && z<=0 && z>=0 }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset (x:xs) = (x:xs):[x]:powerset(xs)
```

Yerman Muncashev

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while i < a.Length {
    out := out - a[i];
    i := i + 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while i >= 0 {
    out := out - a[i];
    i := i - 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b →

(d) `\x l -> x : l ++ l ++ [x]`
a → [b] → [c]

(e) `foldr (const l)`
b → [a] → b

(f) `(."42")`
ill-type

(g) `(.) "42"`
ill-type

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [true] else [false]`

(c) `a -> Maybe b`
`\x -> Just`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`foo f xs ys = f xs ys where`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\(Just x) (`

(h) `Eq a => a -> [a] -> [a]`
`\x xs -> if x == y then xs ++ xs else xs`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

y	x	z	m
0	4	2	2
4	4	1	2
8	4	0	2

$(m-z)\cdot x = y$

$m = 2$

y	x	z	m
0	9	3	3
9	9	2	3
18	9	1	3

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ (m-m)·(m·m) = 0                          }}
  y := 0;
{{ (m-m)·(m·m) = y                          }}
  var x := m * m;
{{ (m-m)·x = y                              }}
  var z := m;
{{ (m-z)·x = y                              }}
  while (z > 0) {
{{ (m-z)·x = y ∧ z>0                        }} ->>
{{ (m-(z-1))·x = y+x                        }}
    z := z - 1;
{{ (m-z)·x = y+x                            }}
    y := y + x;
{{ (m-z)·x = y                              }}
}
{{ (m-z)·x = y ∧ ¬(z>0)                     }} ->>
{{ y = m * m * m }}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [][] = []
weave (x:xs) (y:ys) = x : y :(weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset (x:xs) =
```

Yash Thakur
118533551

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i := a.Length-1;
  while (i >= 0) {
    out := a[i] - out;
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var i := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
(Eq a, Show a, Show (a, a)) ⇒ a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const l)`
ill-typed

(f) `(,"42")`
a → (a, String)

(g) `(,) "42"`
a → (String, a)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
(Ord a, Num a) ⇒ [a] → [a]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
[Maybe Integer]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> (fmap f a) <*> b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f g a -> filter g (map f a)`

(g) `Maybe a -> (a -> b) -> [(a,b)]`
`\a f -> foldMap (\a -> map (a,) (f a)) a`

(h) `Eq a => a -> [a] -> [a]`
`\a l -> filter (a ==) l`

(i) `Show a => [a] -> IO String`
`\a -> putStrLn (foldMap show a)`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a, b) f -> f a b`

(k) `((a,b) -> c) -> c`
`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules: write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m ≥ 0                                                    }}
  y := 0;
{{ y - m*m = m*m*(m-m-1) ∧ m ≥ 0                            }}
  var x := m * m;
{{ y - x = x*(m-m-1) ∧ m ≥ 0 ∧ x = m*m                      }}
  var z := m;
{{ y - x = x*(m-z-1) ∧ z ≥ 0 ∧ x = m*m                      }}
  while (z > 0) {
{{ y - x = x*(m-z-1) ∧ z ≥ 0 ∧ x = m*m                      }} ->>
{{ y = x*(m-z) ∧ z ≥ 0 ∧ x = m*m                            }}
    z := z - 1;
{{ y = x*(m-z-1) ∧ z ≥ 0 ∧ x = m*m                          }}
    y := y + x;
{{ y - x = x*(m-z-1) ∧ z ≥ 0 ∧ x = m*m                      }}
  }
{{ y - x = x*(m-z-1) ∧ z = 0 ∧ x = m*m                      }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = x:y: weave xs ys
weave _ _ = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) =
  let p = powerset xs in
  p ++ (map (x:) p)
```

Matthew Mac

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
var i : int := a.Length - 1
out := 0;
while (i >= 0) {
    out := out - a[i];
    i := i - 1;
}
}
```

```
method f2(a:array<int>) returns (out:int) {
var i : int := 0;
out := 0;
while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
}
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a , b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [b] → [b]

(e) `foldr (const l)`
(a → b → b) → b → [a] → b

(f) `(,"42")`
a → b → (a , b)

(g) `(,) "42"`
a → b → (a , b)

(h) `reverse . reverse`
([a] → [a]) → [a]

(i) `\l -> filter (< 42) (l ++ l)`
(a → b → c) → [a]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
[Nothing] → [Nothing]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x → if x then [x] else [False]`

(c) `a -> Maybe b`
`\x → Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f (n:ns) (c:cs) → [f n c]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x y z → Maybe (x y z)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\x y (z:zs) → if (y (x z)) then [x z] else x z : [x z]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x y → [(x, y x)]`

(h) `Eq a => a -> [a] -> [a]`
`\x y → x : y`

(i) `Show a => [a] -> IO String`
`\x → show x`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f → f x y`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m * m * m && m >= 0                                  }}
  y := 0;
{{ y = m * m * m && m >= 0                                  }}
  var x := m * m;
{{ y = x * m && m >= 0                                      }}
  var z := m;
{{ y = x * m && z >= 0                                      }}
  while (z > 0) {
{{ y = x * m && z >= 0 && m >= 0                            }} ->>
{{ y + x = x * m && z - 1 >= 0                              }}
    z := z - 1;
{{ y + x = x * m && z >= 0                                  }}
    y := y + x;
{{ y = x * m && z >= 0                                      }}
}
{{ y = x * m && !(z > 0) && z >= 0                          }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = x : y : weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset x = [x ++ powersetHelper x]

powersetHelper [] = []
powersetHelper [x:xs] = map (\i -> i : [x]) xs ++ powersetHelper xs
```

Koki Matsuoka

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i := a.Length-1;
    out := 0;
    while (i>=0) {
        out := out - a[i];
        i := i-1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i := 0;
    out := 0;
    while (i< a.Length) {
        out := out - a[i]
        i := i+1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)` a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
Bool → Bool → String

(d) `\x l -> x : (l ++ (l ++ [x]))`
a → [a] → [a]

(e) `foldr (const l)`
b → [a] → b

(f) `(."42")`
ill typed

(g) `(.) "42"`
ill typed

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[int] → [int]

(j) `let f x = x in (f 'a', f True)`
a → (String, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
≈ f a → f b
ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x, True]`

(c) `a -> Maybe b`
`\a -> Maybe 1`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f a b -> zipWith f a b`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> if (isJust b) && (isJust a) then Maybe (f a b) else Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f g a -> map f a`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\ma f -> case ma of`
`Nothing = []`
`Just a = zip (maybeToList ma) (f a)`

(h) `Eq a => a -> [a] -> [a]`
`\a as -> if a == head as then (a:as) else as`

(i) `Show a => [a] -> IO String`
`\a -> foldr (\x ac -> putStrLn x) putStrLn "" a`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`undefined`
or `(b -> c) -> (a -> b) -> a -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules: write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with `->>`. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                          }}
  y := 0;
{{                                                          }}
  var x := m * m;
{{                                                          }}
  var z := m;
{{                                                          }}
  while (Z > 0) {
{{                                                          }} ->>
{{                                                          }}
    z := z - 1;
{{                                                          }}
    y := y + x;
{{                                                          }}
}
{{                                                          }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x : y : (weave ys xs)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

1,2,3
1 2,3
2 3

1:2,3
ps([1,2]) [1,3] [2,3]

1,2,3

```
powerset l =
  foldr (\x acc -> if notElem x acc && notElem x
                   (powerset (delete x l)) then
                     x:powerset(delete x l) ++ acc
                   else powerset(delete x l) ++ acc)
        [l] l
```

CMSC 433: Midterm Exam (Spring 2024)

Jugraj Singh UID: 118867663

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.Length-1;
  var final: int := 0;
  while (i >=0){
      final := final - a[i];
      i:= i-1;
  }
  return final;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := 0;
  var final: int := 0;
  while (i < a.length){
      final := final - a[i];
      i := i+1;
  }
  return final;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → a * b

(c) `\x y -> if x == y then show x else show (x,y)`

Eq a => a → a → String

(d) `\x l -> x : l ++ l ++ [x]`

a → [a] → [a]

(e) `foldr (const 1)`

~~ill-typed const takes two~~ Foldable a => b → a → b

(f) (,"42")

ill-typed

(g) (,) "42"

b → (Char, b)

(h) `reverse . reverse`

[a] → [a] ~~ill-typed~~

(i) `\l -> filter (< 42) (l ++ l)`

Ord a => [a] → [a]

(j) `let f x = x in (f 'a', f True)`

Char, Bool

(k) `fmap (fmap (+1)) [Nothing]`

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [False]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f (x:xs) (y:ys) -> if (f x y) then [(f 1 'a')] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f (Just a) (Just b) -> Just (f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 (h:t) -> if (f2 (f1 h)) then [(f1 h)] else (map f1 t)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\(Just x) f ->`

(h) `Eq a => a -> [a] -> [a]`
`\a (x:xs) -> if a==x then [x] else xs`

*(i) `Show a => [a] -> IO String`
`\x:xs => show x >> return ()`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules: write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m*m*m == 0 + (m*(m*m)) && m*m >=0 }}
  y := 0;
{{ m*m*m == y + (m*(m*m)) && m*m >=0 }}
  var x := m * m;
{{ m*m*m == y + (x*(m*m)) && (m*m) >=0 }}
  var z := m;
{{ m*m*m == y + (x*z) && z >=0 }}
  while (z > 0) {
{{ m*m*m == y + (x*z) && z >0 && z>=0 }} ->>
{{ m*m*m == (y+x) + (x*(z-1)) && (z-1)>=0 }}
    z := z - 1;
{{ m*m*m == (y+x) + (x*z) && z>=0 }}
    y := y + x;
{{ m*m*m == y + (x*z) && z>=0 }}
  }
{{ m*m*m == y + (x*z) && z>=0 && !(z>0) }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = x:y:(weave xs ys)
weave _ _ = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset x:xs = [x]:xs:(x:xs):powerset xs
```

Jocelyn Cabrera
117232363

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0
  var i : int := a.Length - 1

  while i >= 0 {
    out := a[i] - (out);
    i := i - 1;

  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0
  var i : int := 0
  var k : int := a.Length - 1
  while i <= k {
    out := out - a[i];

    i := i + 1;
  }
}
```

show :: a → string

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x` a → a

Example answer:
`a -> a`

(b) `\x y -> (x,y)` a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)` a → b → string

(d) `\x l -> x : l ++ l ++ [x]` a → [a] → [a]

(e) `foldr (const 1)` (a → b → b) → b → [a] → [b] (a → b → a) (b → a)

(f) `(,"42")` ill typed — therefore ill typed

(g) `(,) "42"` b → (int, b)

(h) `reverse . reverse` [a] → [a]

(i) `\l -> filter (< 42) (1 : l)` [a] → [a]

(j) `let f x = x in (f 'a', f True)` ill typed

(k) `fmap (fmap (+1)) [Nothing]` ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int` (+1)
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]` `\x → x : []`

(c) `a -> Maybe b` undefined

(d) `(Int -> Char -> Eool) -> [Int] -> [Char] -> [Bool]`

```
func f (x:xs) (y:ys) =
    (f x y) : func f xs ys
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
func f x y = case x of
    Nothing → Nothing
    Just a → case y of
        Nothing → Nothing
        Just b → Just f a b
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
func f g ls =
    foldr (\elem acc →
        (f elem) : acc) []
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
func x f = case x of
    Nothing → []
    Just a → (a, (head (f a))) : []
```

(h) `Eq a => a -> [a] -> [a]`
let x = delete

(i) `Show a => [a] -> IO String`
undefined

(j) `(a,b) -> (a -> b -> c) -> c`

```
func (x,y) f = f x y
```

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m*m*m == m*m*m }}
  y := 0;
{{ y == m*m*m - z*z*z }}
  var x := m * m;
{{ y == m*m*m - z*z*z && x == m*m }}
  var z := m;
{{ y == m*m*m - z*z*z && x == m*m && z == m }}
  while (z > 0)
{{ y == m*m*m - z*z*z && x == m*m && z >= 0 }}
  {
    z := z - 1;
{{ y == m*m*m - (z+1)*(z+1)*(z+1) && x == m*m && z >= 0 }}
    y := y + x;
{{ y == m*m*m - z*z*z - 3*z*z - 3*z - 1 && x == m*m && z >= 0 }}
  }
{{ y == m*m*m - z*z*z && z <= 0 && x == m*m }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

weave ls1 ls2 = Aux ls1 ls2 True

Aux (x:xs) (y:ys) z = if z
then x : Aux xs (y:ys) (not z)
else y : Aux (x:xs) ys (not z)

Aux [] (y:ys) z = y : Aux [] ys (not z)

Aux (x:xs) [] z = [] // never reaches!

Aux [] [] z = [] // end case

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

powerset ls = Aux ls ls where

Aux (x:xs) (y:ys) =

fun one :)

[1,2,3]	[1,2,3]
[1,2,3]	[]
[1,2]	[3]
[1,3]	[2]
[2,3]	[1]
[1]	[2,3]
[2]	[1,3]
[3]	[1,2]
[]	[1,2,3]

-1 - -3 =
-1 + 3 = 2

((0 - 1) - 2) - 3) = -6
(1 - (2 - (0 - 3) = 2

Bradley
1 - 2 = -1 Ireland

2 - 3 = -1
1 - -1

3 - 0

CMSC 433: Midterm Exam (Spring 2024)

(1 - (2 - (3 - 0)
2 -1 3

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```
2

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```
-6

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := 0
  out := 0
  while (i < a.length) {
    out = out - a[i]
    i = i + 1
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := a.length - 1
  out := 0
  while (i ≥ 0) {
    out = a[i] - out
    i = i - 1
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> IO ()

(d) `\x l -> x : l ++ l ++ [x]`
[a] -> [a] -> [a]

(e) `foldr (const 1)`
ill-typed const a -> b -> a foldr is a -> b -> b

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
ill-typed

(h) `reverse . reverse`
ill-typed

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
a -> b -> f a -> F b

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if`

(c) `a -> Maybe b`
`\x -> Just x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`Just $ zipWith`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`Just`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                          }}
  y := 0;
{{                                                          }}
  var x := m * m;
{{                                                          }}
  var z := m;
{{                                                          }}
  while (z > 0) {
{{                                                          }} ->>
{{                                                          }}
    z := z - 1;
{{                                                          }}
    y := y + x;
{{                                                          }}
  }
{{                                                          }} ->>
{{ y = m * m * m}}
```

1:4:2:5:3:6

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave x  y = case (x,y) of
  (x:xt, y:yt) = x : y : weave xt yt
```

tails [1,2,3] = [[1,2,3], [2,3], [3], []]

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset x = powAux x [[]]
  powAux x acc pos=case x of
X:xt => powAux:xt      x : tails x : inits x : take pos x :
take 1 2               drop pos x : acc (pos + 1)
drop 2 3        [] => nub acc
```

Note this is all one line

Evgenia Zoulis
117116144

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a] → [a]

(e) `foldr (const l)`
a → b → (a → b → a)

(f) `(,"42")`
(a, String)

(g) `(,) "42"`
ill-typed

(h) `reverse . reverse`
[a] → [a] → [a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
a → (a → Bool) → [a] → [a]

(j) `let f x = x in (f 'a', f True)`
(a → (a, b)) → (Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed (i don't think the +1 works on Nothing)

Evyenia Zoulis
1171116144

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
foo x = [x]

(c) `a -> Maybe b`
foo x = Nothing

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
zipwith 2 'a' false

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
foo, f = f in f (maybe c)

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
foo x y = if y in

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
\x -> zip x y

(h) `Eq a => a -> [a] -> [a]`
delete 2 [1,2]

(i) `Show a => [a] -> IO String`
foo = put "hello" get x

(j) `(a,b) -> (a -> b -> c) -> c`
foo (a,b) = c a b

(k) `((a,b) -> c) -> c`
foo (a,b) in foo

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m*m*m>0 && m*m*m=y+(z*x) }}
  y := 0;
{{ m*m*m = 0+(z*x)                }}
  var x := m * m;
{{ x*m = 0+(z*x)                  }}
  var z := m;
{{ z*x = 0+(z*x) && z*x=m*m*m     }}
  while (Z > 0) {
{{ z!=0 && y>=0 && m*m*m=x*z       }} ->>
{{ z>0 &&                          }}
    z := z - 1;
{{ m=z+1 && x=m*m &&               }}
    y := y + x;
{{ y=x*(m-z) && x=m*m && z<=0      }}
}
{{ y=x*m && !(z>0) && x=m*m        }} ->>
{{ y = m * m  * m}}
```

Evyenia Zoulis
11711644

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x : y : weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) = [x] : [powerset xs] ++ [[x:xs]]
```

Josh Horner
117925464

CMSC 433: Midterm Exam (Spring 2024)

1, 2, 3, 4

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

1 - (2 - (3 - (4 - (0))))

(((0 - 1) - 2) - 3) - 4

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    out := 0;
    var i:int := a.Length-1;
    while (i >= 0) {
        out := a[i] - out;
        i := i - 1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    out := 0;
    var i:int := 0;
    while (i < a.Length) {
        out := out - a[i];
        i := i+1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
(Show a, Eq a) => a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const !)`
ill-typed ithink

(f) `(."42")`
a -> (a, String)

(g) `(,) "42"`
a -> (String, a)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
Ord a => [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
Num a => [Maybe a]

Josh Horner
1179254

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\a -> [a]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`liftA2`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
f m g [] = []
f m g (a:as) = if (g (m a)) then (m a) : f m g as
               else f m g as
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
f Nothing abs' = []
f (Just a) abs' = zip (repeat a) (abs' a)
```

(h) `Eq a => a -> [a] -> [a]`
`delete`

(i) `Show a => [a] -> IO String`
`\as -> putStrLn (concat (map show as))`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
1 {{ (y + x*z == m*m*m && z >= 0)[z := m][x := m*m][y := 0] }}
  y := 0;
2 {{ (y + x*z == m*m*m && z >= 0)[z := m][x := m*m] }}
  var x := m * m;
3 {{ (y + x*z == m*m*m && z >= 0)[z := m] }}
  var z := m;
4 {{ y + x*z == m*m*m && z >= 0 }}
  while (z > 0) {
5 {{ y + x*z == m*m*m && z >= 0 && z > 0 }} ->>
6 {{ (y + x*z == m*m*m && z >= 0)[y := y + x][z := z - 1] }}
    z := z - 1;
7 {{ (y + x*z == m*m*m && z >= 0)[y := y + x] }}
    y := y + x;
8 {{ y + x*z == m*m*m && z >= 0 }}
  }
9 {{ y + x*z == m*m*m && z >= 0 && !(z > 0) }} ->>
  {{ y = m * m  * m}}
```

y + x*z == m*m*m && z >= 0

z >= 0

1
2
3
4
5 y + x*z == m*m*m && z >= 0 && z > 0
6
7
8
9 y + x*z == m*m*m && z >= 0 && !(z > 0) => y + x*0 == m*m*m

Josh Horner
11742544

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (a:as) (b:bs) = a:b:(weave as bs)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

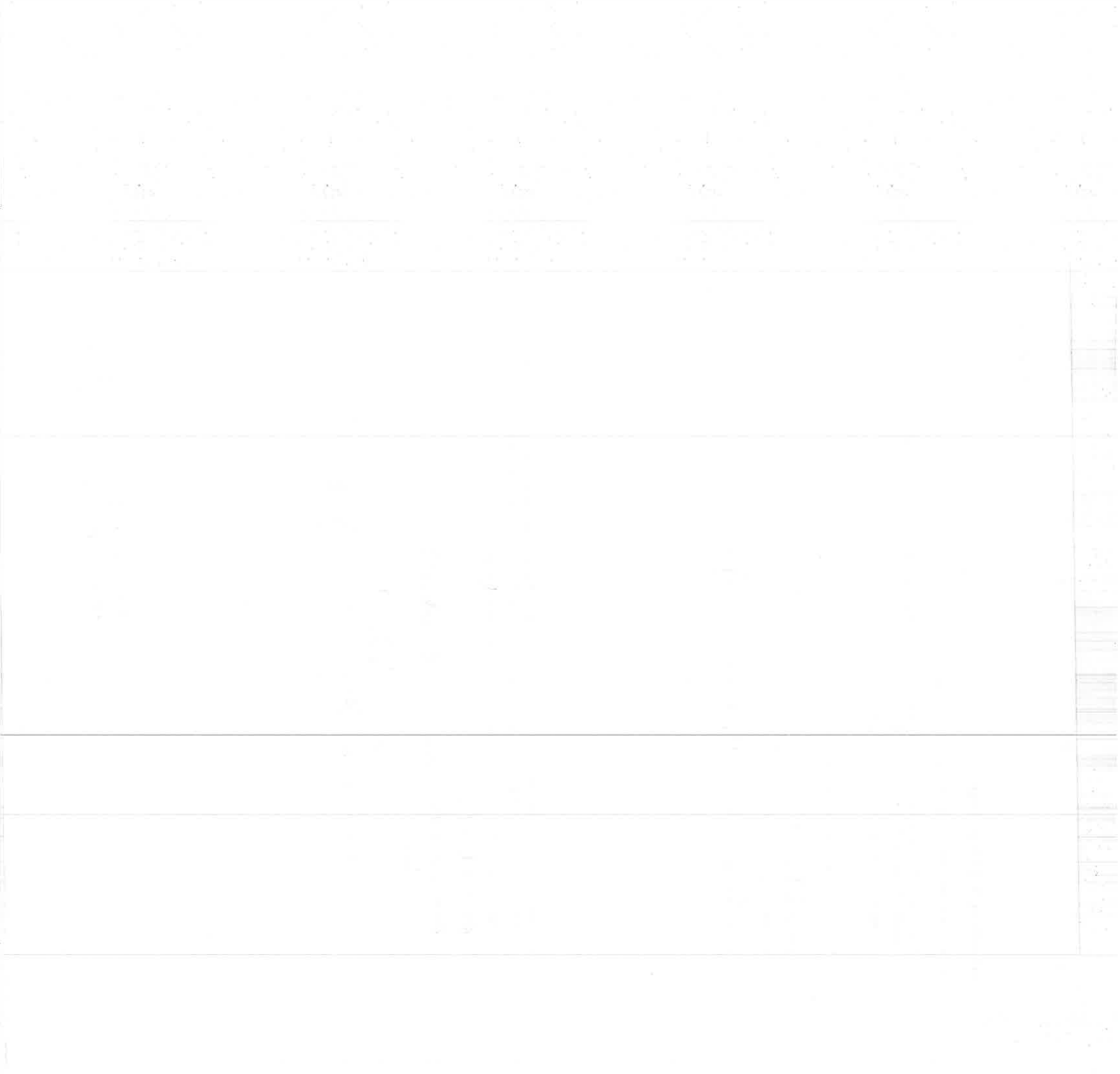
```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset as = nub (takeall as (length as))
    where
      takeall as 0 = as:[[]]:[]
      takeall as n = (powerset (drop n as)) ++ (takeall as (n-1))
```

[1,2,3,4]

powerset [] = []
powerset as ==

takeall as 0 == as:[[]]:[]
takeall as n == powerset (drop n as) ++
takeall



Ziqi (Steven) Yang

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    out := 0;
    var i := a.Length;
    while i > 0 {
        i := i - 1;
        out := a[i] - out;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    out := 0;
    var i := 0;
    while i < a.Length {
        out := out - a[i];
        i := i + 1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
(Eq a, Show a, Show (a,a)) ⇒ a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
ill-typed

(f) `(,"42")`
a → (a, String)

(g) `(,) "42"`
a → (String, a)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (1 ++ l)`
(Num a, Ord a) ⇒ [a] → [a]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
Num a ⇒ [a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
pure

(c) `a -> Maybe b`
const Nothing

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
zipWith

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
liftA2

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f g -> filter g . map f`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`maybe (const []) (\x f -> map (x,) $ f x)`

(h) `Eq a => a -> [a] -> [a]`
`filter . (==)`

(i) `Show a => [a] -> IO String`
`pure . concat . map show`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`($ (undefined, undefined))`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{m>=0&& 0==m*m*(m-m)&& m*m==m*m }}
  y := 0;
{{m>=0&& y==m*m*(m-m)&& m*m==m*m }}
  var x := m * m;
{{m>=0&& y==x*(m-m)&& x==m*m }}
  var z := m;
{{z>=0&& y==x*(m-z)&&x==m*m }}
  while (z > 0) {
{{z>=0&& y==x*(m-z)&&x==m*m&& z>0 }} ->>
{{z-1>=0&& y+x==x*(m-(z-1))&&x==m*m }}
    z := z - 1;
{{z>=0&& y+x==x*(m-z)&& x==m*m }}
    y := y + x;
{{z>=0&& y==x*(m-z)&& x==m*m }}
}
{{ z>=0&& y==x*(m-z)&& x==m*m&&!(z>0) }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave = concat . zipWith (\x y -> [x, y])
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) = map (x:) (powerset xs) ++ powerset xs
```

Thomas Jacob Hval

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.length
  out := 0;
  while(i > 0){
    out = a[i] - out;
    i = i - 1
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0
  out = 0;
  while(i < a.length){
    out = out - a[i];
    i = i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const 1)`
ill-types

(f) `(,"42")`
a -> (String, a)

(g) `(,) "42"`
ill-types

(h) `reverse . reverse`
a -> b

(i) `\l -> filter (< 42) (l ++ l)`
[a] -> [a]

(j) `let f x = x in (f 'a', f True)`
ill-types

(k) `fmap (fmap (+1)) [Nothing]`
ill-types

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [x]`

(c) `a -> Maybe b`
undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\x y z -> [(x (head y) (head z))]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x y z -> case y of Just a => (case z of just b -> x a b`
`Nothing => Nothing Nothing => Nothing)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`x y z -> let b = (x (head z)) in if y b then [b] else [b]`

(g) `Maybe a -> (a -> [a]) -> [(a,b)]`
undefined

(h) `Eq a => a -> [a] -> [a]`
`\x y -> if x == x then y else y`

(i) `Show a => [a] -> IO String`
`\x -> print ln (Show x)`

(j) `(a,b) -> (a -> b -> c) -> c`
`\x y => let (a,b) = x in (y a b)`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m*m*(m-m)                                  }}
  y := 0;
{{ y = m*m*(m-m                                   }}
  var x := m * m;
{{ y = x*(m-m)                                    }}
  var z := m;
{{ y = x*(m-z)                                    }}
  while (z > 0) {
{{ y = x*(m-z)  && z > 0                          }} ->>
{{ y+x = x*(m-z-1)                                }}
    z := z - 1;
{{ y+x = x*(m-z)                                  }}
    y := y + x;
{{ y = x*(m-z)                                    }}
  }
{{ y = x*(m-z) && !(z > 0)                        }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave (xs:xt) (ys:yt) = xs:(ys:(weave xt yt))
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset (xs:xt) = (helper xs xt) ++ (powerset xt) where
helper el [] = []
helper el [single] = (el:[single]):[el]:(helper el [])
helper el (xs:xt) = (el:(xs:xt)):[el,xs]:(helper el xt))
```